

UNIVERSITATEA DIN BUCUREȘTI



**FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ**

SPECIALIZAREA INFORMATICĂ

Lucrare de licență

TravelPal

Absolvent

Aldea Alexia-Elena

Coordonator științific

Prof. Mihai Prunescu

București, iunie 2024

Rezumat

În ultimii ani, călătoriile în alte țări au devenit tot mai dorite de tot mai mulți oameni, astfel că numărul aplicațiilor mobile destinate acestui lucru a crescut. Însă, problema lor este că sunt repetitive și niciuna nu le oferă suport și persoanelor care nu au partener de călătorie și nici nu vor să meargă în vacanță singuri.

Această lucrare are ca obiectiv prezentarea unei noi idei de aplicații, care îmbină în mod perfect căutarea biletelor de avion cu conectarea oamenilor singuri, despre care se știe că au 2 lucruri în comun: o destinație și pasiunea pentru călătorit. Pe parcursul lucrării sunt descrise atât noțiuni teoretice, cât și tehnice, care constituie procesul de creare al aplicației TravelPal.

În Capitolul 1, am prezentat o introducere în aplicație și în ceea ce vrea aceasta să facă, am prezentat cu ce vine nou aplicația și câteva aplicații care se aseamănă cu o parte din TravelPal.

În Capitolul 2, am prezentat arhitectura folosită în aplicație și conceptele ei, cum se construiește și cum se folosește aceasta și de ce este bine să o folosim, cu ce ne ajută aceasta codul - arhitectura MVVM.

În Capitolul 3, am vorbit despre principiile de Clean Code, ce înseamnă, cum se folosește, cum impactează codul.

În Capitolul 4, am prezentat tehnologiile folosite, noțiuni teoretice și cum le-am folosit pe parcursul aplicației.

În Capitolul 5 se află prezentarea aplicației TravelPal în detaliu. De la business logic și până la posibile direcții de dezvoltare ale aplicației, se află totul acolo, detaliat.

În final, în ultimul capitol am prezentat câteva concluzii trase de pe parcursul implementării aplicației și de ce este necesară aplicația TravelPal, de ce este o aplicație mobilă.

Abstract

In recent years, travel to other countries has become increasingly desired by more and more people, so the number of mobile applications designed for this purpose has grown. However, their issue is that they are repetitive, and none of them offer support to individuals who do not have a travel companion and do not want to go on vacation alone.

The objective of this paper is to present a new idea for an application that seamlessly combines the search for airline tickets with connecting solo travelers, who are known to have two things in common: a destination and a passion for travel. Throughout the paper, both theoretical and technical concepts are described, constituting the process of creating the TravelPal application.

In Chapter 1, an introduction to the application and what it aims to achieve is presented. The new features of the application and a few similar applications to TravelPal are also discussed.

In Chapter 2, the architecture used in the application and its concepts are presented, along with how it is built and used, and why it is beneficial to use it—the MVVM architecture and its impact on code.

In Chapter 3, the principles of Clean Code are discussed—what it means, how it is used, and its impact on code.

In Chapter 4, the technologies used, theoretical concepts, and how they were applied throughout the application are presented.

Chapter 5 provides a detailed presentation of the TravelPal application, covering everything from business logic to possible directions for the application's development.

Finally, in the last chapter, some conclusions drawn from the implementation of the application are presented, discussing why the TravelPal application is necessary and why it is a mobile application.

Cuprins

1	Introducere.....	5
1.1	Încadrarea în context: nevoia pentru aplicație.....	5
1.2	Aplicații similare.....	6
1.3	Evidențierea caracteristicilor diferite ale aplicației.....	7
2	Arhitectura MVVM.....	9
2.1	Prezentare.....	9
2.2	Avantaje.....	14
2.3	Convenții.....	15
2.4	Principiile S.O.L.I.D.....	16
3	Clean Code.....	20
4	Prezentarea tehnologiilor folosite.....	24
4.1	Xcode.....	24
4.2	SwiftUI.....	30
4.3	Combine.....	32
4.4	Data persistence.....	39
4.5	State management.....	41
4.6	Firebase.....	45
4.7	Figma.....	46
5	Prezentarea aplicației TravelPal.....	47
5.1	Business logic.....	47

5.2 Structura aplicației TravelPal.....	48
5.3 Funcționalități.....	48
5.4 Posibile direcții de dezvoltare.....	56
6 Concluzii.....	58
Bibliografie.....	59

1. Introducere

1.1 Încadrarea în context: nevoia pentru aplicație

TravelPal a venit ca o soluție a problemei tot mai des întâlnite: "Vreau să călătoresc. Cu cine pot face acest lucru?".

Un studiu realizat de Forbes[1] arată că, anual, din ce în ce mai mulți oameni își doresc să viziteze locuri noi. Din acest punct, intervine problema menționată anterior. TravelPal, prin funcționalitatea sa unică, abordează această problemă într-un mod complet inedit și inovator. Utilizatorului i se expune o listă generoasă de zboruri, iar acesta își poate achiziționa unul sau mai multe zboruri. În momentul în care un utilizator își achiziționează un bilet de avion pentru o anumită destinație, este adăugat automat într-un chat cu toți ceilalți utilizatori care au fost, sunt sau vor merge în aceeași destinație. Acest lucru creează o comunitate de călători care pot împărtăși sfaturi, recomandări și experiențe despre destinația respectivă.

Această funcționalitate nu numai că facilitează schimbul de informații între călători, dar creează și o rețea de suport pentru cei care călătoresc singuri sau pentru prima dată într-o anumită locație. Prin aceste caracteristici, TravelPal nu este doar o altă aplicație de călătorie, ci un partener de încredere pentru orice călătorie. Într-o lume în care călătoriile devin din ce în ce mai frecvente, TravelPal este aici pentru a face experiența de călătorie mai sigură, mai ușoară și mai plăcută.

În prezent, călătoriile au devenit o parte esențială în viața cotidiană. Într-o lume globalizată, oamenii sunt mai dornici ca niciodată să exploreze noi culturi, să cunoască noi oameni și să experimenteze noi aventuri. De asemenea, s-a constatat[2] că această activitate are o multitudine de avantaje, ce-i permit călătorului să se dezvolte într-un mod deosebit :

- Descoperim noi lucruri despre noi înșine: Călătoritul ne scoate din zona de confort, din rutina pe care ne-o creăm în viața de zi cu zi. Datorită numărului infinit de scenarii care pot apărea în timpul unei vacanțe, suntem obligați în mod automat să ne descurcăm în orice situație ar putea apărea. Astfel, descoperim că ne putem descurca în orice situație.
- Învățăm să socializăm cu oricine: Cu cât întâlnim tot mai mulți oameni, mai ales în călătorii, cu atât ne îmbunătățim abilitatea de a comunica și, de asemenea, avem ocazia să facem schimb de experiențe cu oameni din diferite locuri ale lumii -> un lucru care ne poate deschide orizonturi cu totul noi.

- Primim o educație diferită față de cea din cărți: Cititul ne oferă un spectru limitat de înțelegere, pe când, atunci când avem parte într-o călătorie noi experiențe, descoperim cu adevărat, de exemplu, anumite culturi, religii sau stiluri de viață. Călătoritul ne educă într-un mod special, deoarece învățăm lucruri noi prin practică, lucruri pe care, poate, în viața de zi cu zi nu le-am întâlni.

Cu toate acestea, călătoriile pot fi, de asemenea, o provocare. De la planificarea itinerariului la găsirea celor mai bune oferte de zbor și cazare, de la înțelegerea culturii locale la navigarea într-un oraș necunoscut - toate acestea pot fi dificile, mai ales atunci când călătorești singur.

Aici intervine TravelPal. Prin crearea unei comunități de călători care împărtășesc aceeași destinație, TravelPal transformă provocările călătoriilor în oportunități. În loc să călătorești singur, acum poți să te conectezi cu alți călători, să împărtășești sfaturi și recomandări, și să te bucuri de suportul unei comunități. De asemenea, oamenii, dintotdeauna, au simțit nevoia să fie alături de cineva, în orice situație. De exemplu, la birou, la școală, la facultate, oamenii lucrează în echipă, împreună, se strâng împreună în același loc pentru a atinge un obiectiv. Acest lucru se aplică și la călătorii, oamenii preferând să fie alături de cineva când descoperă această lume plină de necunoscut. Însă, cu TravelPal, nimeni nu mai trebuie să călătorească singur.

1.2 Aplicații similare

În prezent, există numeroase aplicații care au funcționalități similare cu TravelPal, precum:

- Chat Random: Aceasta este o aplicație de chat care permite utilizatorilor să comunice cu alții la întâmplare. Deși nu se concentrează pe călătorii, ar putea fi folosită de călători pentru a intra în contact cu alții care se află în aceeași locație

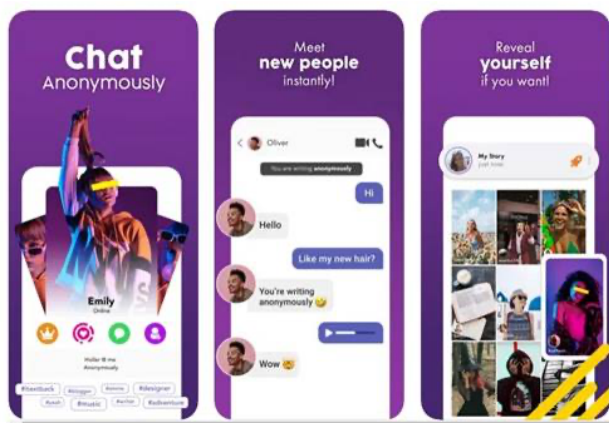


Fig 1.2.1, exemplu aplicație similară

- Google Chat: Google Chat permite utilizatorilor să creeze diferite camere de chat și să

adauge persoane în funcție de nevoi. Cu toate acestea, nu adaugă automat oamenii în camere de chat pe baza destinațiilor lor de călătorie



Figura 1.2.2, exemplu de aplicație similară

La momentul actual, există numeroase aplicații destinate achiziționării de bilete de avion sau destinate creării unor chat-uri, aleatoare sau nu. Însă, aceste tipuri de aplicații sunt separate, astfel că nu există niciuna care să le îmbine: achiziționarea unui bilet de avion și generarea unui chat, cu mai mulți oameni, care merg în aceeași destinație.

1.3 Evidențierea caracteristicilor diferite ale aplicației

Deși aplicația TravelPal a pornit dintr-o joacă și din dorința mea de a îmbina comunicarea între oameni cu achiziționarea biletelor de avion, aceasta prezintă niște funcționalități care chiar o pot face specială și, care de asemenea, rezolvă o problemă actuală în lume: "Eu cu cine călătoresc?":

- deși există aplicații care permit utilizatorilor să comunice cu alți călători, acestea nu oferă automat adăugarea utilizatorilor într-un chat pe baza destinației lor de călătorie. TravelPal se diferențiază prin această funcționalitate unică, care facilitează interacțiunea între călători cu interese comune.
- în timp ce alte aplicații de chat pot fi utilizate și de călători, TravelPal este special concepută pentru a conecta călătorii și a le oferi un mediu dedicat pentru a împărtăși experiențe, sfaturi și informații despre destinații.
- TravelPal oferă o experiență personalizată prin adăugarea automată a utilizatorilor într-un chat cu alți călători care au fost, sunt sau vor merge în aceeași destinație. Aceasta asigură relevanța și interacțiunea cu persoanele care au experiență sau interes în aceeași

locație.

- TravelPal se remarcă prin originalitatea și inovația sa în domeniul aplicațiilor de călătorie. Funcționalitatea sa unică și abordarea sa inovatoare fac din TravelPal o aplicație distinctă și atractivă pentru călători.
- posibilitatea de a obține sfaturi și informații de la alți călători, de a găsi parteneri de călătorie sau de a crea legături cu oameni cu interese similare în timpul călătoriilor.

2. Arhitectura MVVM

2.1 Prezentare

- Cu ce ajută aplicația mobilă existența unei arhitecturi cât mai performante?[3]

O arhitectură eficientă pentru o aplicație mobilă este aceea care promovează principii și modele de programare solide, cum ar fi SOLID, KISS și care asigură existența multiplor straturi de responsabilitate în componente. Respectarea acestor criterii facilitează dezvoltarea rapidă și ușurează întreținerea ulterioară, economisind astfel resurse valoroase de timp și bani.

Alegerea judicioasă a unei arhitecturi, în combinație cu tehnologia specifică platformei (Swift pentru iOS, Kotlin pentru Android), este cea mai adecvată pentru a aborda în mod eficient problemele complexe de business în cadrul proiectelor mobile. Grație acestei abordări, obiectivul general poate fi realizat într-o manieră unificată, ceea ce va conduce la o mai mare eficiență a timpului. Unul dintre aspectele fundamentale ale unei arhitecturi bune este separarea straturilor de responsabilitate. O astfel de divizare va fi esențială pentru a limita contextul și amplitudinea erorilor.

Adoptarea unei arhitecturi este esențială în dezvoltarea aplicațiilor, mai ales pentru cele cu complexitate ridicată. Fără o arhitectură bine definită, codul interfeței utilizator (UI) poate fi amestecat cu cel de comunicare cu backend-ul / baza de date, ceea ce poate duce la un proiect dificil de gestionat pe termen lung. Pentru a preveni această problemă, se pot utiliza diverse arhitecturi, precum MVC (Model-View-Controller), MVP (Model-View-Presenter), MVVM (Model-View-ViewModel) și altele. Fiecare dintre acestea are propriile sale avantaje, motiv pentru care companiile de top își bazează proiectele pe structuri solide care să le optimizeze aplicațiile.

Arhitectura MVVM (Fig. 2.1.1) este una dintre cele mai recente și inovatoare arhitecturi de acest gen. A fost concepută cu scopul de a separa interfața grafică de codul de backend, fiind cunoscută și sub denumirea de MVB (Model-View-Binder). Aceasta se datorează faptului că, odată cu separarea interfeței grafice și a codului de backend, a fost necesar un mediator pentru a facilita comunicarea între acestea. Acest mediator interacționează cu ambele componente, transmițând informații între UI și backend, menținând în același timp separarea acestora.

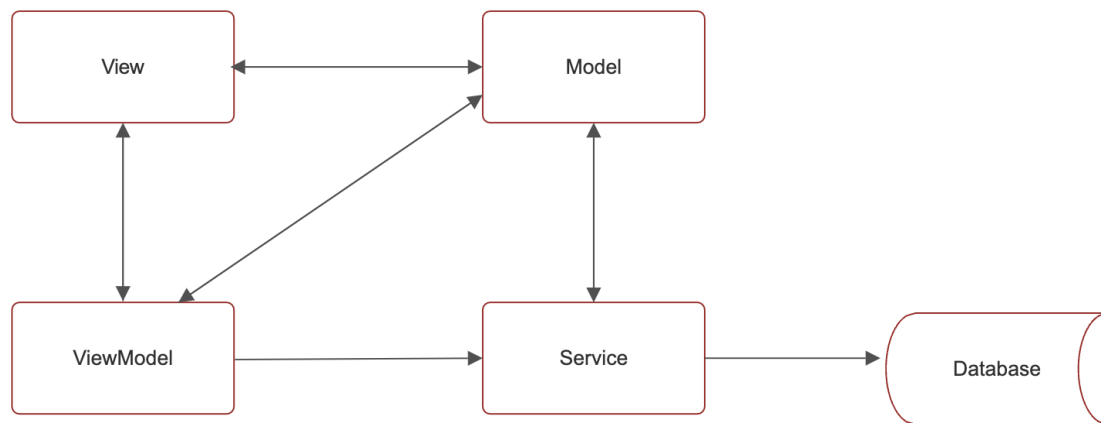


Figura 2.1.1, arhitectura MVVM

În figura de mai sus, se poate observa cum funcționează arhitectura MVVM. Informațiile din backend vin printr-un API, iar acest API este apoi chemat în Service. În Service se află, în general, funcții care ne ajută să manipulăm mai ușor datele care vin din API sau care trebuie trimise la API. Din Service, datele pleacă în ViewModel, dar sub forma unui Model. Datele nu pot pleca din Service oricum, trebuie așezate în funcție de Modelele de date. ViewModelul primește doar datele care îi sunt necesare din Service. La baza lui se pot afla, de asemenea, funcții care să ne ajute să umblăm mai bine cu datele, funcții care pot fi folosite ulterior în View.

Componentele arhitecturii MVVM au următoarele roluri:

- **Modelul** reprezintă logica de business a aplicației, incluzând datele și modul în care acestea sunt procesate (Fig. 2.1.2).. O modularizare eficientă a modelului contribuie la ușurința de întreținere a codului, îmbunătățind în același timp lizibilitatea acestuia. Prin definiția sa, modelul se referă la datele și logica de business a unei aplicații. Reprezintă datele și operațiunile specifice domeniului care sunt independente de orice implementare specifică a interfeței utilizator (UI).

Modelul încapsulează datele și oferă metode pentru accesarea și manipularea acestor date. Poate include clase, structuri sau alte tipuri de date care reprezintă entitățile aplicației, cum ar fi conturile de utilizatori, produsele sau orice alte date relevante.

Modelul este responsabil pentru menținerea integrității și consistenței datelor, efectuarea de calcule sau validări și interacțiunea cu sursele de date, cum ar fi baze de date sau servicii web.

Caracteristica principală a Modelului este că nu are nicio cunoștință sau dependență de UI sau stratul de prezentare. Este conceput pentru a fi decuplat de componentele interfeței utilizator, permițându-i să fie reutilizabil și independent de orice tehnologie specifică de UI.

```
struct Airport: Identifiable, Decodable {  
    var id: UUID = UUID()  
    let name: String  
    let iataCode: String  
    let icaoCode: String  
    let latitude: Double  
    let longitude: Double  
    let countryCode: String  
}
```

Figura 2.1.2, Model de date

- **View-ul** reprezintă interacțiunea utilizatorului cu aplicația, incluzând toate aspectele legate de prezentare (Fig. 2.1.3-2.1.4).. Acesta ar trebui să fie proiectat având în vedere experiența utilizatorului din toate perspectivele. De asemenea, View-ul reprezintă componentele interfeței utilizator (UI) ale unei aplicații. Este responsabilă pentru prezentarea datelor către utilizator și capturarea interacțiunilor acestuia.

View-ul este locul unde sunt definite elementele UI, cum ar fi butoanele, casetele de text, meniurile și alte controale. Oferă o reprezentare vizuală a datelor și expune mecanismele necesare pentru a afișa și interacționa cu aceste date.

În MVVM, View-ul este pasiv și nu conține nicio logică de business sau cod de manipulare a datelor. În schimb, se leagă de proprietățile și comenzile furnizate de ViewModel pentru a afișa și actualiza datele. Prin legarea datelor, View-ul reflectă automat schimbările din ViewModel, iar interacțiunile utilizatorului sunt gestionate de comenzi definite în ViewModel. Prin separarea UI de datele și logica de business subiacente, modelul MVVM promovează reutilizabilitatea și testabilitatea View-ului. Permite designerilor și dezvoltatorilor să lucreze independent la designul și comportamentul UI, fără a-l lega strâns de date sau logica aplicației.

Pe scurt, View-ul este responsabil pentru redarea UI și interacțiunea cu utilizatorul. Se leagă de proprietățile și comenzile expuse de ViewModel pentru a afișa și actualiza datele, păstrând în același timp logica UI separată de logica de business.

```
struct OnBoardingScreenView: View {  
    @EnvironmentObject private var navigation: Navigation  
    @StateObject var viewModel = OnBoardingViewModel()  
}
```



```

var body: some View {
    VStack(alignment: .leading, spacing: 0) {
        Image("logo_app1")
            .resizable()
            .frame(width: 153, height: 27)
            .padding(.bottom, 44)
            .padding(.top, 24)
            .padding(.horizontal, 24)

        Text("Meet and Explore with TravelPal")
            .font(.Poppins.bold(size: 28))
            .multilineTextAlignment(.leading)
            .padding(.bottom, 16)
            .padding(.horizontal, 24)

        Text("Join conversations with fellow travelers who share your passion for your dream holiday
destination")
            .font(.Poppins.regular(size: 14))
            .multilineTextAlignment(.leading)
            .padding(.bottom, 24)
            .padding(.horizontal, 24)

        ZStack {
            Image("bg-onboarding")
                .resizable()

            VStack(spacing: 0) {
                Spacer()

                RedButtonView(text: "Access my TravelPal Account") {
                    viewModel.userDefaultsService.setOnboarding(onboardingIsOver: true)
                    navigation.push(LoginScreenView().asDestination(), animated: true)
                }.padding(.bottom, 14)

                BlackButtonView(text: "Continue as Guest") {
                    viewModel.userDefaultsService.setOnboarding(onboardingIsOver: true)
                    navigation.replaceNavigationStack([MainScreenView().asDestination()], animated: true)
                }.padding(.bottom, 16)
            }.padding(.horizontal, 24)
        }
    }
}

```

Figurile 2.1.3-2.1.4, View

- **ViewModel-ul**, așa cum sugerează și numele, face legătura între UI și Model (Fig. 2.1.5). Acționează ca un intermediar între componentele Model și View. Furnizează datele și comportamentul necesare pentru View, facilitând în același timp comunicarea și sincronizarea între View și Model.

Scopul principal al ViewModel-ului este de a abstractiza View-ul de la Modelul subiacent și de a oferi o reprezentare a datelor care este adaptată specific nevoilor View-ului. Acesta expune proprietăți, comenzi și alte mecanisme de legare a datelor la care View-ul se poate lega, permițând UI să se actualizeze automat atunci când datele subiacente se modifică. ViewModel-ul încapsulează de asemenea logica de prezentare și gestionează interacțiunile utilizatorului. Conține metode și comenzi care sunt invocate de View ca răspuns la acțiunile utilizatorului, cum ar fi clicurile pe butoane sau modificările de intrare. ViewModel-ul traduce apoi aceste acțiuni în operațiuni pe Model, actualizând datele și menținând consistența stării aplicației.

Unul dintre beneficiile cheie ale ViewModel-ului este că promovează separarea preocupărilor. Permite dezvoltatorilor să proiecteze și să testeze UI independent de datele și logica de business subiacente, îmbunătățind întreținerea și organizarea codului. În plus, ViewModel-ul poate oferi date și comportament în timpul proiectării pentru o mai bună vizualizare și prototipare a UI.

Este important de notat că ViewModel-ul nu are nicio dependență directă de cadrul UI specific utilizat. Ar trebui să fie o reprezentare agnostică de platformă a datelor și comportamentelor necesare pentru View, făcându-l reutilizabil în diferite tehnologii UI.

În general, ViewModel-ul servește ca punte între Model și View, oferind legarea datelor, logica de prezentare și gestionarea interacțiunilor utilizatorului, menținând în același timp UI separat de datele și logica de business subiacente.

```
class ChatViewModel: BaseViewModel {
    @Published var user: User?
    @Published var userCities: [City] = []
    @Published var countUsers: Int = -1

    var userService = UserService.shared
    var firestoreService = FirestoreService.shared

    override init() {
        super.init()
        userService.user
            .sink { _ in

            } receiveValue: { user in
                self.user = user
            } .store(in: &bag)

        firestoreService.userCities
            .sink { _ in

            } receiveValue: { cities in
                self.userCities = cities
            } .store(in: &bag)
```



Figura 2.1.5, ViewModel

2.2 Avantaje

Arhitectura MVVM poate fi considerată o evoluție a pattern-urilor MVP sau MVC, aducând unele avantaje subtile, dar cu un impact semnificativ.

Într-un proiect construit cu pattern-ul Model-View-Controller, adesea apare întrebarea cu privire la locul în care trebuie plasat codul. Codul care nu se potrivește în model sau view este de obicei plasat în controller. Aceasta duce la clase de controller care sunt dificil de testat și întreținut. Pattern-ul Model-View-ViewModel rezolvă această problemă prin adăugarea de view modele. Acestea transformă datele provenite din model în informații care pot fi utilizate de view. Astfel, controller-ul nu mai are această responsabilitate.

De asemenea, controller-ul este greu de testat din cauza dependenței sale de view. Prin mutarea manipulării datelor în view model, testarea devine mult mai ușoară, deoarece acesta nu necesită o referință la obiectul care îl deține. Fără a mai depinde de view, testarea poate fi realizată pe un obiect independent.

Responsabilitățile controller-ului sunt reduse la interacțiunea dintre view și model, construind puntea între cele două componente. View modelul oferă o interfață transparentă pentru controller-ul de view, pe care acesta o utilizează pentru a popula view-ul și a interacționa cu model-ul. Aceasta duce la o comunicare transparentă între cele patru componente ale aplicației.

Alte avantaje ale arhitecturii MVVM includ[4]:

- Reutilizarea eficientă a codului și dezvoltarea paralelă: ViewModel-ul în MVVM poate fi reutilizat în mai multe view-uri sau chiar pe diferite platforme. Această reutilizare poate duce la economii semnificative de timp și la o consistență crescută a aplicației. În plus, deoarece modelul, view-ul și view modelul sunt decuplate, diferite echipe pot lucra la ele în paralel, accelerând procesul de dezvoltare.
- Legarea datelor: Legarea datelor între view și view model permite propagarea automată a modificărilor, ceea ce înseamnă că atunci când datele se schimbă într-un loc, ele sunt actualizate în toate locurile în care sunt referite. Acest lucru duce la mai puțin cod similar și la o singură sursă, un cod reutilizabil, făcând codul mai curat și mai ușor de înțeles.
- Separarea clară a preocupărilor: Arhitectura MVVM promovează separarea clară a

preocupărilor între UI, logica de business și straturile de date. Aceasta facilitează testarea și întreținerea codului, deoarece fiecare componentă poate fi testată și modificată independent, fără a afecta celelalte componente.

- Prototipare și vizualizare mai ușoară: ViewModel-ul poate oferi date și comportament în timpul proiectării, permițând o mai bună vizualizare și prototipare a UI-ului. Acest lucru facilitează colaborarea între designeri și dezvoltatori și permite o validare rapidă a conceptelor de UI.

2.3 Convenții

În arhitectura MVVM în Swift, există câteva convenții importante de urmat:

1. View model-ul nu ar trebui să importe nicio clasă din framework-ul UIKit (precum UILabel, UIButton, UIImageView etc.). Acest lucru ajută la menținerea independenței viewModel-ului față de platforma specifică și facilitează reutilizarea acestuia în diferite tehnologii UI.
2. În view, nu ar trebui să existe logica aplicației. Mutarea condiționărilor, loop-urilor sau a oricărui alt cod care manipulează datele în viewModel ajută la testarea mai ușoară a codului și la menținerea modularității acestuia. În view trebuie să se afle o referință către viewModel și, în rest, elemente folosite strict pentru UI.
3. Nu se recomandă să se transmită view-uri către viewModel. View-urile pot fi distruse și recreate, iar view model-ul ar trebui să fie conștient doar de model și să nu depindă de ciclul de viață al view-urilor. Transmiterea view-urilor către view model poate duce la probleme precum data leak sau crash-uri ale aplicației.
4. În schimb, view-urile pot observa direct schimbările din view model. În view model, se pot folosi variabile de tip ObservableObject sau Published pentru a notifica view-urile despre modificările datelor. Aceste variabile pot fi observate de către view-uri prin intermediul property wrapper-ului @ObservedObject sau @StateObject. Astfel, se poate profita de avantajele arhitecturii MVVM și observarea schimbărilor poate fi utilă în situații precum rotirea ecranului.
5. Repository-ul de date ar trebui să fie singurul punct direct de intrare a datelor. Repository-ul este responsabil pentru gestionarea datelor și poate avea acces la datele remote, locale sau cache. Acesta oferă o interfață simplificată pentru a obține și a actualiza datele și ajută la separarea clară între UI și backend.

Aceste convenții ajută la menținerea unei structuri clare și modulare în arhitectura MVVM în Swift și facilitează testarea și întreținerea codului.

2.4 Principiile S.O.L.I.D.

Principiile de proiectare SOLID[5] sunt un set de recomandări care ne ajută să gestionăm problemele de proiectare a software-ului. Aceste principii reprezintă conceptele de bază pe care orice cod scris în spiritul programării orientate pe obiecte (POO) ar trebui să le îndeplinească. Sunt principii generale, folosite în orice limbaj de programare.

Introduse de Robert C. Martin și dezvoltate ulterior de Michael Feathers, principiile SOLID reprezintă o colecție de reguli care trebuie respectate pentru a crea software ușor de întreținut, inteligibil și flexibil. Aceste principii au ca scop reducerea dependențelor, astfel încât programatorii să poată schimba o parte a software-ului fără a afecta restul.

Deși respectarea principiilor SOLID aduce numeroase beneficii, este important de menționat că poate duce la cod mai lung și mai complex. Procesul de proiectare poate fi extins, iar dezvoltarea poate deveni mai dificilă. Cu toate acestea, timpul și efortul suplimentar sunt justificate de faptul că software-ul rezultat va fi mai ușor de întreținut, testat și extins.

A. Single responsibility

Principiul Responsabilității Unice (SRP) afirmă că fiecare clasă sau modul ar trebui să aibă o singură responsabilitate și să se concentreze exclusiv pe aceasta. Acest principiu ne îndeamnă să evităm cuplarea între funcționalități diferite și să încapsulăm în mod complet responsabilitatea într-o singură clasă sau modul.

Practic, acest principiu ne îndeamnă să ne asigurăm că fiecare clasă sau modul rezolvă sau tratează doar un singur scop. Astfel, evităm situațiile în care o clasă sau funcție trebuie să se ocupe de mai multe funcționalități, ceea ce ar putea duce la o cuplare excesivă între acestea.

Aplicând acest principiu, fiecare clasă sau modul ar trebui să aibă responsabilitatea asupra unei singure părți a funcționalității software-ului și să își îndeplinească această responsabilitate în întregime. Toate serviciile și funcționalitățile oferite de clasă ar trebui să fie aliniate cu această responsabilitate unică.

Deși respectarea acestui principiu poate duce la cod mai lung și mai complex, beneficiile sunt semnificative. Având clase sau module cu responsabilitate unică, vom obține un cod mai ușor de întreținut, testat și extins. De asemenea, vom evita dependențele excesive între funcționalități și vom facilita reutilizarea și modularitatea codului.

B. Open Close

Entitățile software, cum ar fi clasele, modulele și funcțiile, ar trebui să respecte două

principii importante: deschise pentru extindere și închise pentru modificare. Aceste principii ne îndeamnă să proiectăm entități care pot fi extinse fără a modifica codul existent și care nu necesită modificări ale codului care le utilizează.

Principiul deschis pentru extindere ne permite să extindem funcționalitatea unei entități atunci când cerințele aplicației se schimbă. Putem adăuga noi funcționalități sau comportamente la o clasă fără a modifica codul existent. Astfel, putem face clasele mai funcționale și mai adaptabile la schimbările cerințelor. În același timp, comportamentul vechilor metode rămâne neschimbat, iar clasa în sine nu suferă modificări.

Principiul închis pentru modificare ne îndeamnă să evităm modificările codului care utilizează entitățile. După ce am extins comportamentul unei entități, nu ar trebui să fie necesară modificarea codului care folosește acea entitate. Astfel, putem evita introducerea de erori și probleme în codul existent.

Un exemplu practic al aplicării acestor principii este utilizarea claselor abstracte într-o bibliotecă. În loc să modificăm clasele concrete din bibliotecă, putem extinde clasele abstracte pentru a adăuga funcționalități suplimentare. Astfel, putem extinde comportamentul entităților fără a afecta codul existent.

În concluzie, un modul sau o entitate este considerată deschisă pentru extindere atunci când poate fi extinsă fără a modifica codul inițial. În schimb, un modul sau o entitate este considerată închisă pentru modificare atunci când poate fi utilizată de alte module fără a necesita modificări ale codului existent. Aceste principii ne ajută să creăm software flexibil, ușor de întreținut și adaptabil la schimbările cerințelor.

C. Liskov substitution

Dacă pentru fiecare obiect de tip S există un obiect de tip T astfel încât comportamentul programelor definite în T rămâne neschimbat atunci când înlocuim obiectul de tip S cu obiectul de tip T, atunci putem spune că S este un subtip al lui T. Acest principiu ne indică că subclasa trebuie să conțină toate proprietățile superclasei și să poată fi utilizată în locul acesteia. Subclasa nu trebuie să modifice funcționalitățile superclasei, dar poate adăuga funcționalități noi.

Cu toate acestea, acest principiu poate fi încălcat atunci când superclasa apelează un serviciu extern într-o metodă, iar subclasa rescrie complet acea metodă. În această situație, principiul nu mai este respectat.

Principiul substituției Liskov afirmă că orice clasă derivată poate înlocui clasa de bază și codul va continua să funcționeze corect. Dacă S este un subtip al lui T, atunci obiectele

de tip T pot fi înlocuite cu obiecte de tip S fără a afecta funcționalitățile esențiale ale programului.

Acest principiu impune anumite condiții în scrierea metodelor. Argumentele metodelor unui subtip trebuie să fie contravariante, adică să respecte o relație inversă de ordine între părinte și copil. De asemenea, tipurile returnate de metodele unui subtip trebuie să fie covariante. Subtipurile nu ar trebui să arunce excepții, cu excepția cazului în care acele excepții sunt subtipuri ale celor aruncate de metodele tipului părinte. În concluzie, noile clase extinse din clasa de bază trebuie să poată înlocui clasa de bază în toate funcțiile sale fără a fi necesare modificări suplimentare.

D. Interface segregation

Acest principiu, cunoscut și sub numele de Principiul Interfeței Separate, subliniază importanța ca niciun client să nu fie obligat să depindă de metodele pe care nu le utilizează. În loc să avem o interfață mare și complexă, care conține multe metode, dar din care doar câteva sunt folosite de fiecare client, este mai indicat să împărțim acea interfață în interfețe mai mici și mai specifice, numite interfețe de rol. Astfel, fiecare client va trebui să cunoască doar metodele care îi sunt relevante.

Scopul acestui principiu este de a menține un sistem decuplat, ceea ce face ca refactorizarea, modificarea și redistribuirea să fie mai ușoare. Prin împărțirea interfețelor în module mai mici, clasele au opțiunea de a implementa doar interfețele necesare, evitând astfel dependențele inutile.

Principiul Interfeței Separate promovează ideea de a evita forțarea clienților să depindă de metodele pe care nu le folosesc. Acest lucru se realizează prin împărțirea interfețelor în module mai mici, astfel încât clasele să aibă opțiunea de a le implementa sau nu. Această abordare ajută și la decuplarea funcționalităților, ceea ce facilitează re-factorizarea produsului în viitor.

E. Dependency inversion

Acest principiu, cunoscut sub numele de Principiul Inversiunii Dependințelor, se referă la cuplarea dintre diferitele clase sau module dintr-un sistem software. Principiul afirmă că modulele de nivel înalt nu ar trebui să depindă de modulele de nivel scăzut, ci ambele module ar trebui să depindă de abstractizări.

Mai exact, acest principiu promovează ideea că abstracțiile ar trebui să fie dependente de detalii, ci detaliile ar trebui să depindă de abstracții. În cazul în care o clasă are o dependență, aceasta ar trebui să fie o interfață, nu o clasă concretă. Această abordare

încurajează decuplarea claselor și este esențială pentru testarea unitară.

Principiul Inversiunii Dependențelor are ca scop crearea unui sistem software mai flexibil și mai ușor de întreținut. Prin inversarea dependențelor, modulele de nivel înalt devin mai independente de modulele de nivel scăzut, ceea ce facilitează modificările și refactorizările ulterioare. De asemenea, acest principiu permite testarea unitară mai eficientă, deoarece dependențele pot fi înlocuite cu ușurință cu simulacre în timpul testării.

3. Clean Code

De ce este important ca pe tot parcursul dezvoltării să avem un cod curat? Cum ne putem da seama că avem un cod curat? De ce companiile de IT nu acceptă un cod dezordonat?

Pentru a răspunde la aceste întrebări, hai să ne gândim, întâi, ce înseamnă un cod "curat". Este un cod care este ușor de citit de către oricine, ușor de înțeles, nu își schimbă structura și, în același timp, este foarte performant.

Un cod curat este important deoarece are multiple beneficii. În primul rând, un cod curat este mai ușor de întreținut și de modificat în viitor. Datorită structurii clare și a stilului de scriere coerent, este mai ușor pentru dezvoltatori să înțeleagă și să adauge funcționalități noi sau să rezolve probleme. De asemenea, un cod curat este mai ușor de testat, deoarece este mai previzibil și mai ușor de izolat anumite părți pentru testare unitară.

Un alt beneficiu al unui cod curat este că este mai ușor de colaborat cu alți dezvoltatori. Un cod bine organizat și documentat permite altor programatori să se integreze mai ușor în proiect și să lucreze eficient împreună. De asemenea, un cod curat facilitează revizuirea și auditarea de către alți membri ai echipei sau de către clienți.

Există câteva indicii care ne pot ajuta să ne dăm seama că avem un cod curat. În primul rând, un cod curat respectă principiile de proiectare și stilul de scriere convenit în echipă. Este bine documentat, cu comentarii clare și explicative. De asemenea, un cod curat este modular și are funcții sau clase bine definite și specializate. Este lipsit de duplicare de cod și folosește denumiri semnificative pentru variabile, funcții și clase.

Companiile de IT nu acceptă un cod dezordonat deoarece acesta poate duce la probleme grave în dezvoltarea și întreținerea software-ului. Un cod dezordonat poate fi greu de înțeles și de modificat, ceea ce duce la creșterea costurilor și a timpului necesar pentru dezvoltare. De asemenea, un cod dezordonat poate fi mai greu de testat și de deslușit, ceea ce poate duce la apariția de erori și la scăderea calității produsului final. În plus, un cod dezordonat poate afecta colaborarea în echipă și poate crea dificultăți în gestionarea proiectului.

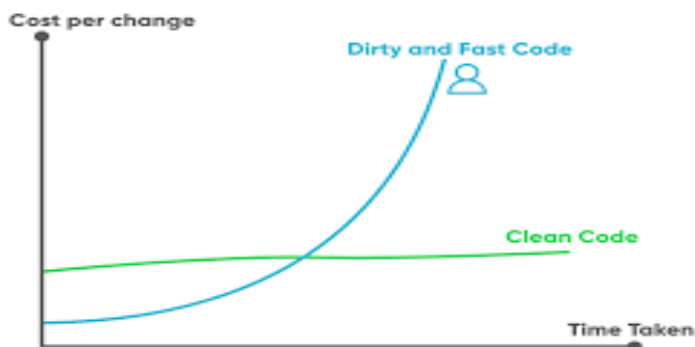


Figura 3.1 Relația dintre productivitate și timp în contextul codului curat/dezordonat¹

Denumiri semnificative

Denumirea variabilelor, a funcțiilor, claselor etc. este una dintre cele mai importante chei pentru a scrie un cod bun. Denumirile trebuie să fie cât mai sugestive, pentru că atunci când altcineva citește codul, să poată înțelege ce face o variabilă, o funcție, o clasă, doar din numele ei.

```
var ceva: String //text
```

Pentru variabila definită mai sus, nu înțelegem din numele ei care este scopul ei. O denumire corectă a acesteia ar fi:

```
var title: String
```

În următorul exemplu, vom analiza aceeași funcție, dar cu denumiri diferite:

```
var languages: [String] = ["romana", "franceza", "engleza"]
var x: Bool = false

func print() {
  for language in languages {
    if language == "araba" {
      x = true
      print(x)
    }
  }
}
```

Codul de mai sus nu este complicat, dar nici nu este foarte intuitiv despre ce se vrea să facă. Spre deosebire de codul de mai jos, unde, funcționalitatea lui este aceeași, dar, în schimb, denumirile sunt diferite, mult mai explicite, astfel că, în orice moment, știm care este scopul oricărei variabile sau a funcției în sine.

¹ Imagine preluată din articolul “Clean Code, Why is it, why it matters?”, autor necunoscut, URL: <http://www.thorstenkranz.de/index.php/software-development>

```
var languages: [String] = ["romana", "franceza", "engleza"]
var wordWasFound: Bool = false

func print() {
    for i in languages {
        if i == "araba" {
            wordWasFound = true
        }
    }
}
```

Funcții

Funcțiile trebuie să fie scurte, să nu ajungă la lungimi de 100 de linii. Acestea ar trebui să aibă un singur scop, foarte bine definit. Definirea funcției ar trebui să îi exprime scopul.

Este important să avem funcții scurte și cu un singur scop bine definit pentru a facilita citirea și înțelegerea codului. Astfel, codul devine mai ușor de întreținut și de modificat în viitor.

De exemplu, în loc să avem o funcție lungă care face mai multe lucruri, este mai bine să avem mai multe funcții mai scurte, fiecare cu un scop specific. Acest lucru face codul mai modular și mai ușor de înțeles.

Un exemplu de funcție cu un scop bine definit și o denumire sugestivă:

```
func calculeazaSuma(a: Int, b: Int) -> Int {

    return a + b
```

} -> Această funcție are un singur scop - să calculeze suma a două numere. Denumirea funcției, "calculeazaSuma", exprimă clar acest scop și face funcția ușor de înțeles.

Alte convenții

- Variabilele se declară toate la început, înainte de a scrie funcțiile.
- Denumirile variabilelor și a funcțiilor trebuie să înceapă cu litera mică, iar dacă o denumire conține mai multe cuvinte, primul cuvânt trebuie scris cu litera mică, iar următoarele cu litera mare.
- Denumirile claselor și a fișierelor trebuie să înceapă cu litera mare, iar dacă acestea conțin mai multe cuvinte, prima literă de la fiecare cuvânt trebuie să fie mare.
- La declararea variabilelor, este de preferat să le declarăm și tipul, chiar dacă le inițializăm pe loc, pentru a elimina viitoarele posibile confuzii.
- Variabilele, funcțiile, clasele, fișierele, elementele de UI nefolosite, dar care apar în cod, ar trebui șterse, deoarece acestea complică înțelegerea și compilarea codului în mod

eficient.

În concluzie, un cod curat este esențial în dezvoltarea software-ului deoarece aduce beneficii în întreținere, colaborare și calitate. Prin respectarea principiilor de proiectare și a unui stil de scriere coerent, putem crea un cod curat care să fie ușor de înțeles, de modificat și de testat. Companiile de IT nu acceptă un cod dezordonat deoarece acesta poate afecta eficiența și calitatea dezvoltării software-ului.

4. Prezentarea tehnologiilor folosite

4.1 Xcode

Xcode este o suită de tool-uri folosite de dezvoltatorii iOS pentru a crea aplicații care pot fi folosite ulterior pe toate dispozitivele Apple[6]. Xcode oferă posibilitatea dezvoltatorilor să creeze, să testeze, să optimizeze și chiar să pună pe Apple Store aplicațiile create în el.

De asemenea, Xcode include un "world-class code editor". Acest lucru înseamnă că, pe măsură ce programatorul scrie codul, poate vedea în timp real, în preview. "World-class code editor" se referă la un software de calitate superioară, pe care programatorii îl folosesc pentru a scrie, a modifica și a analiza codul. Aceste editoare vin adeseori cu funcționalități precum colorarea cuvintelor din cod, completarea automată, instrumente de debugging, etc. Xcode face parte din această categorie de editoare.

Xcode include, de asemenea, numeroase instrumente pentru a testa rapid codul scris. Atunci când își dă run, Xcode deschide automat Simulatorul.

În plus, Xcode vine cu un "Interface Builder" încorporat. Acesta permite dezvoltatorilor să proiecteze și să testeze interfața utilizatorului fără a scrie o linie de cod. Acest instrument face posibilă prototiparea UI în câteva minute, iar apoi conectarea grafică a interfeței la codul sursă în editorul Xcode.

Simulatorul inclus în Xcode este un alt instrument puternic. Acesta permite dezvoltatorilor să testeze aplicațiile pe o varietate de dispozitive și versiuni de sistem de operare, fără a avea nevoie de dispozitivele fizice. Acesta poate simula diferite condiții de mediu și poate ajuta la identificarea și rezolvarea problemelor înainte de lansarea aplicației.

Structura proiectelor

Structura proiectelor în Xcode este simplă: reprezintă un fișier al tuturor fișierelor, informațiilor și pachetelor necesare pentru a construi o aplicație/un software.

- ➔ Project File = aici țin minte toate referințele, dependențele și tot ce este folosit în proiect (aici este creierul proiectului)
- ➔ Targets = spunem despre un target că se referă la un produs, o aplicație care trebuie să fie construită. Aici se găsesc instrucțiunile despre cum trebuie să fie construită aplicația. Aceste instrucțiuni sunt sub forma unor fișiere, iar fiecare fișier poate avea propriile setări.

- Workspaces = spunem despre un workspace că este un loc ce conține mai multe proiecte care au resurse în comun.
- Schemes = o schemă este o colecție de targeturi care trebuie să fie construite, o configurație folosită atunci când aplicația este construită și un set de teste care trebuie executate atunci când aplicația este testată.
- Build Settings = aici se află instrucțiunile necesare pentru ca aplicația să poată fi construită.
- Source Files = aici se află toate fișierele cu cod.
- Resource Files = fișiere folosite în proiect, precum imagini, culori, storyboard-uri.
- Frameworks and Libraries = aici se află toate librăriile pe care le putem folosi în cod.
- Groups = sunt folosite pentru a organiza fișierele.
- Dependencies = referințe la alte proiecte pe care se bazează proiectul meu.

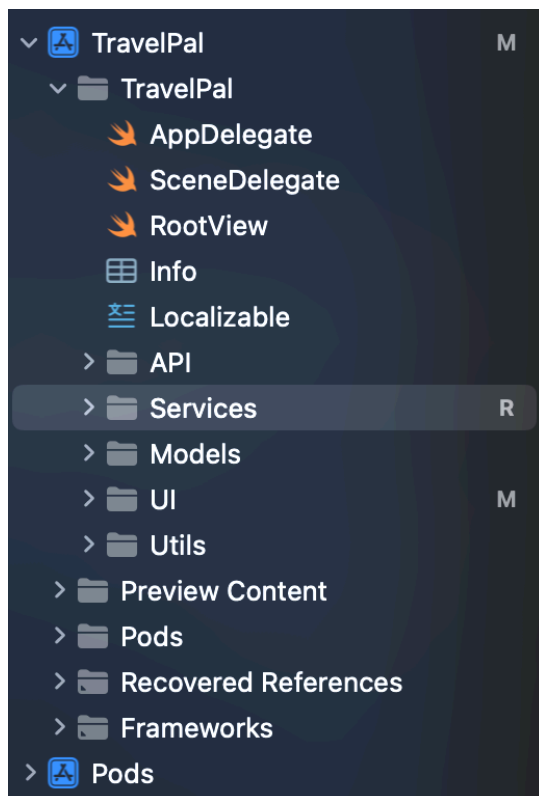


Figura 4.4.1.1. Structura unui proiect în Xcode

După cum se poate observa în imagine, există un singur fișier, care conține tot ce este necesar pentru o aplicație realizată în Xcode.

În primul folder TravelPal se introduce codul sursă al aplicației, organizat și el, la rândul lui, în diferite fișiere, pentru a respecta arhitectura MVVM și pentru o lizibilitate mult mai clară a proiectului și a fișierelor pe care le conține. Componentele acestui folder sunt:

- Info = conține setările aplicației (de exemplu, dacă vrem să cerem permisiune la locație în aplicație, sau la camera video, dacă vrem să avem un splashscreen, etc)
- RootView = este fișierul responsabil de navigarea principală din aplicație.
- API = este un folder ce conține toate apelurile la API-urile folosite în acest proiect (le voi detalia în următorul capitol, unde voi vorbi strict despre aplicație).

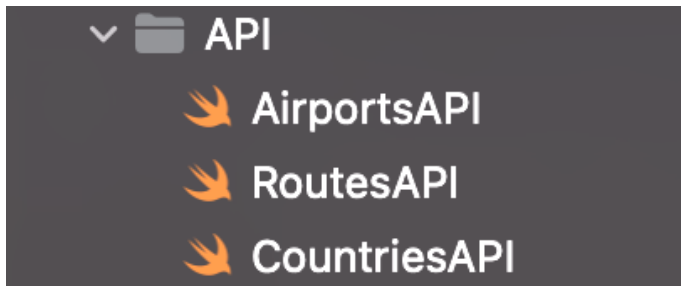


Figura 4.4.1.2 Folder pentru Api call-uri

- Service = este un folder ce conține toate funcționalitățile principale ale aplicației. Există câte un Service special pentru fiecare API, dar, de asemenea, pot exista și Service-uri care se ocupă cu diferite alte lucruri, utile în toată aplicația. În service se află funcționalități care sunt necesare în mai multe zone din cod și, astfel, service-ul face legătura între ele. De exemplu, aveam nevoie să știu în mai multe puncte ale aplicației dacă este vreun utilizator conectat, pentru a ști ce să afișez în funcție de acest lucru. Astfel, am folosit un Service numit UserService.

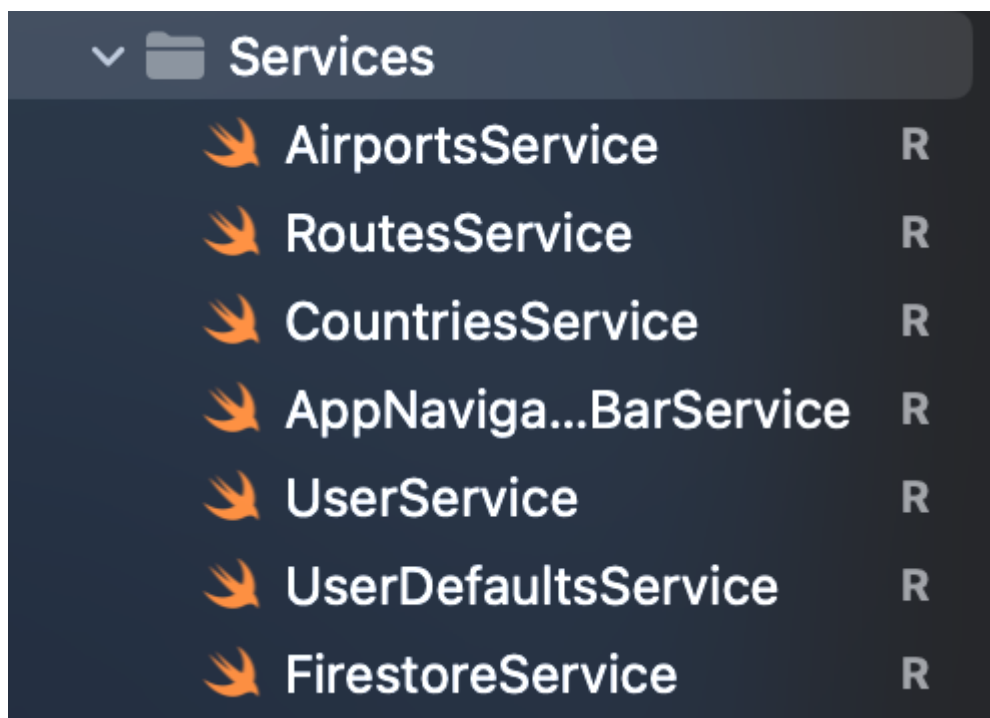


Figura 4.4.1.3 Folder pentru Service

➔ Models = În acest folder se află toate tipurile de modele de date. De exemplu, pentru fiecare aeroport venit de la Service (care a preluat datele de la API), avem nevoie de un obiect numit "Aeroport", care va avea ca și attribute toate informațiile pe care trebuie să le stocăm pentru un aeroport, informații care sunt venite de la API.

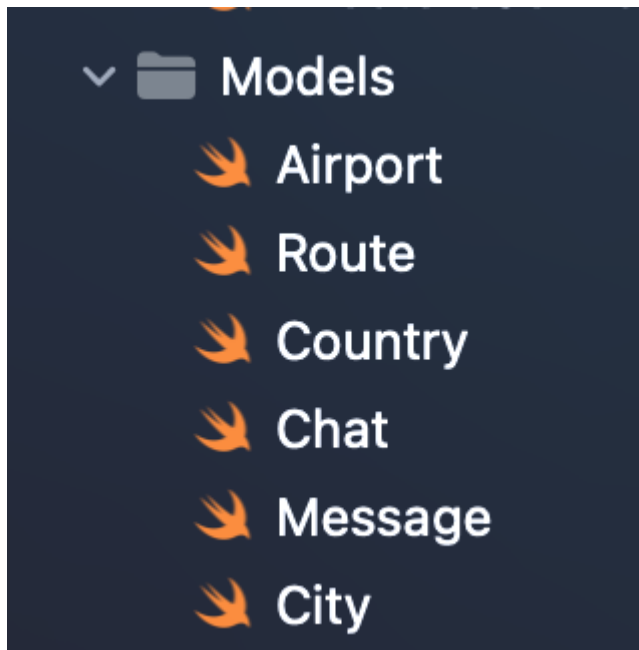


Figura 4.4.1.4 Folder pentru modele de date

➔ UI = În acest fișier vom avea toate componentele care țin de UI: ViewModele și View-uri. Pentru ordine și lizibilitate, am organizat toate componentele acestui folder în sub-foldere mai mici, dedicate câte unui ecran în parte.

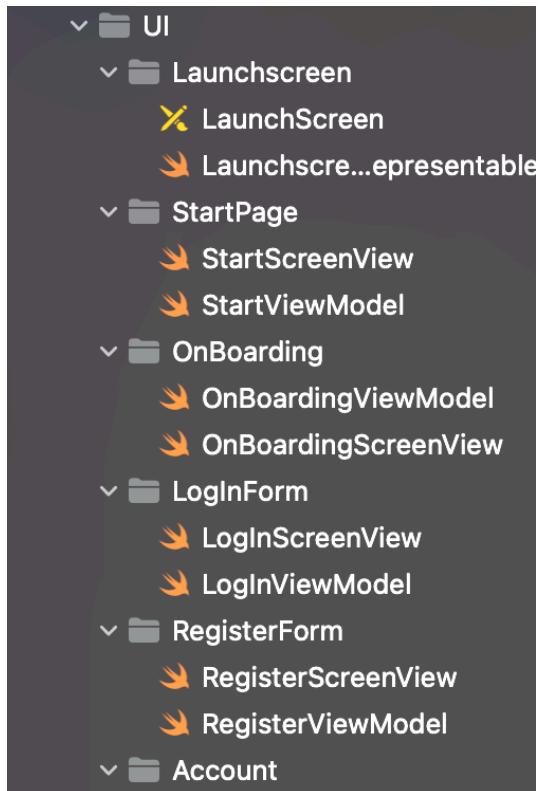


Figura 4.4.1.5 Folder pentru UI

→ Utils = Am dedicat acest folder tuturor componentelor reutilizabile, Assets-urilor (adică pozelor, culorilor). În general, după cum îi spune și numele, vrem ca în acest folder să avem toate fișierele care nu au ce căuta în cele de mai sus.

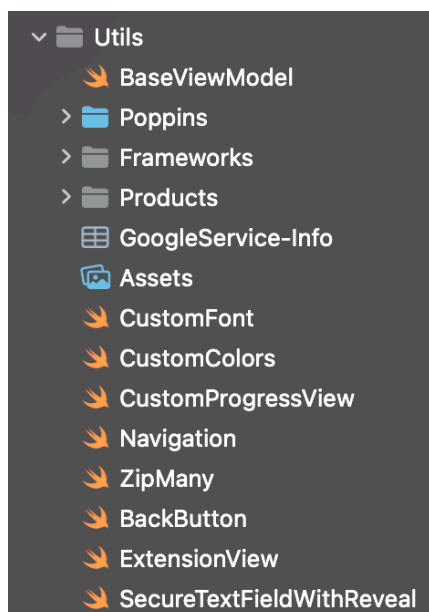


Figura 4.4.1.6 Folder pentru Utils

→ SceneDelegate = Acest fișier joacă unul dintre cele mai importante roluri într-un proiect de Swift. Acesta se ocupă cu gestionarea modului în care interfața este afișată utilizatorului și, de asemenea, cum răspunde la modificarea stării aplicației. Începând de la iOS 13, SceneDelegate a fost făcut să funcționeze pe mai multe ecrane (scene), făcând

astfel ca aplicațiile care necesită folosirea acțiunilor "multitasking" să funcționeze mult mai bine.

O scenă reprezintă o instanță a aplicației, o instanță a interfeței cu care utilizatorul interacționează. Introducerea acestor scene în iOS a fost un pas major pentru a eficientiza multitasking-ul pe telefoane și pe tablete. SceneDelegate este responsabil de gestionarea acestor scene, el se ocupă de modul în care ele sunt create, activate/dezactivate și terminate și, de asemenea, el se ocupă atunci când utilizatorul schimbă scenele. Câteva dintre responsabilitățile fișierului "SceneDelegate":

- Gestionează fereastra: El creează și gestionează scenele aplicației, prin funcția `scene`. În această metodă, o nouă fereastră "UIWindow" este creată și i se setează și rădăcina (rootViewController).

```
func scene(_ scene: UIScene, willConnectTo session: UISceneSession, options connectionOptions:
UIScene.ConnectionOptions) {

    if let windowScene = scene as? UIWindowScene {
        let window = UIWindow(windowScene: windowScene)
        navigation = Navigation(root: StartScreenView().asDestination())
        window.rootViewController = UIHostingController(rootView: RootView(navigation: navigation))
        self.window = window
        window.makeKeyAndVisible()
    }
}
```

Figura 4.4.1.7 Metoda scene_

- Răspunde la Lifecycle Events: SceneDelegate conține numeroase metode care sunt folosite în diferite momente ale vieții unei scene. De exemplu, metoda "sceneDidBecomeActive(_:)" este utilizată atunci când o scenă devine activă, iar "sceneDidEnterBackground(_:)" este utilizată atunci când o scenă este rulată în fundal.

```
func sceneDidDisconnect(_ scene: UIScene) {}
func sceneDidBecomeActive(_ scene: UIScene) {}
func sceneWillResignActive(_ scene: UIScene) {}
func sceneWillEnterForeground(_ scene: UIScene) {}
func sceneDidEnterBackground(_ scene: UIScene) {}
```

Figura 4.4.1.8 Diferite LifeCycle Events folosite in TravelPal

- Gestionează restaurarea stării aplicației: SceneDelegate poate ajuta la restaurarea stării aplicației, prin a salva starea curentă atunci când o scenă este deconectată. Apoi, restaurează starea atunci când scena se reconectează.
- Se poate ocupa de mai multe scene: În caz că aplicația suportă mai multe ecrane (de exemplu, pe iPad), SceneDelegate ajută la gestionarea multiplelor scene, prin

a crea o nouă sesiune de scene pentru fiecare ecran. În final, SceneDelegate joacă unul dintre cele mai importante roluri pentru gestionarea interfeței utilizatorului în aplicație. Acesta răspunde la schimbările stărilor din aplicație și se ocupă de mai multe scene, în paralel.

→ AppDelegate = Este cealaltă componentă foarte importantă a unui proiect în Swift, deoarece coordonează multe comportamente abstracte ale aplicației. Este un delegat al obiectului "UIApplication" și este responsabil cu ciclul de viață al evenimentelor aplicației. AppDelegate este punctul principal de intrare în aplicație (fiind marcat la începutul codului cu @main). În proiectul meu, AppDelegate este responsabil de configurarea Firebase-ului (vom detalia puțin mai târziu despre acesta) atunci când aplicația se pornește și definește configurarea noilor scene.

4.2 SwiftUI

Pentru a realiza aplicația TravelPal, am scris cod folosind librăriile din SwiftUI. Însă, pentru a putea vorbi despre SwiftUI, este necesar să ne întoarcem puțin în timp și să vorbim despre strămoșii lui SwiftUI[7].

Scara ierarhică ar fi: Objective-C, UIKit și apoi Swift/SwiftUI. Objective-C a fost primul limbaj de programare folosit pentru a crea aplicații în iOS, atunci când a apărut primul iPhone. În acest timp, a fost introdusă și librărie numită UIKit, cu ajutorul căreia programatorul putea construi interfețe pentru iOS. UIKit se bazează pe Objective-C, oferind o mare cantitate de unelte ce ne ajută să construim interfețe complexe. De la apariția lui iPhone OS2, UIKit a evoluat în mod continuu, ajungând astfel, în prezent, la un nivel foarte puternic, care îi aduce programatorului libertatea să creeze aproape orice în aplicațiile pentru iOS.

În anul 2014, compania Apple a introdus limbajul de programare numit Swift, limbaj ce este menit să fie mai sigur, mai rapid și mai permisiv față de Objective-C. La baza sa, Swift conține principiile din Objective-C, respectiv principii ale programării orientate pe obiecte. De atunci, programatorii s-au folosit de limbajul Swift și de librărie UIKit pentru a crea aplicații tot mai puternice și avansate pentru iOS.

În anul 2019, compania Apple a introdus o nouă librărie, ce avea să fie un înlocuitor de viitor al UIKit, numită SwiftUI. SwiftUI este o librărie ce are la bază elemente de UIKit și se folosește de limbajul Swift. Spre deosebire de UIKit, SwiftUI se folosește strict de limbajul Swift.

De ce este atât de importantă apariția lui SwiftUI? Prin introducerea acestei librării, programatorii au o libertate mult mai mare de a crea UI-ul unei aplicații. SwiftUI oferă o sintaxă declarativă, ce permite programatorilor să descrie cum ar trebui să arate UI-ul, cum ar trebui să se comporte, ce stări ar trebui să aibă, etc. Practic, programatorul poate decide orice ce întâmplă în UI, datorită acestei librării. Aici intervine diferența majoră dintre SwiftUI și UIKit: în UIKit stilul de programare este unul imperativ, unde programatorii trebuie să descrie cum trebuie să se întâmple schimbările din UI.

SwiftUI este realizat pentru a funcționa pe toate platformele Apple, astfel încât programatorii să fie capabili să construiască aplicații pentru orice platformă (iOS, macOS, watchOS, tvOS), folosind un singur set de unelte. De la introducerea sa, SwiftUI a captat atenția tot mai multor programatori, astfel că, în prezent, majoritatea aplicațiilor mobile și a programatorilor fac tranziția de la folosirea UIKit la folosirea SwiftUI.

Această scară ierarhică de la Objective-C la SwiftUI reprezintă răspunsul companiei Apple la evoluția continuă a tehnologiei și a nevoii dezvoltatorilor de iOS să realizeze cât mai multe lucruri pe platformele ce folosesc sistemul de operare OS. SwiftUI este un tool mult mai modern și mai puternic față de tot ce a existat până acum pentru dezvoltarea aplicațiilor în iOS.

Este important de menționat că, deși SwiftUI este un tool puternic și modern, tranziția de la UIKit la SwiftUI nu este un proces care se întâmplă peste noapte. Multe aplicații încă se bazează pe UIKit pentru anumite părți ale UI-ului, iar dezvoltatorii pot folosi atât UIKit, cât și SwiftUI împreună în aceeași aplicație. Acest lucru permite o tranziție mai lină și mai flexibilă către SwiftUI. În plus, SwiftUI continuă să evolueze și să se îmbunătățească cu fiecare nouă versiune a iOS, adăugând funcționalități și îmbunătățiri care îl fac și mai atractiv pentru dezvoltatori.

Un alt aspect important al SwiftUI este că acesta facilitează dezvoltarea de aplicații cross-platform. În trecut, dezvoltarea de aplicații pentru iOS, macOS, watchOS și tvOS necesita cunoștințe despre diferite seturi de unelte și abordări de programare. Cu SwiftUI, dezvoltatorii pot folosi același set de unelte și aceeași sintaxă pentru a crea UI-uri pentru toate platformele

Apple. Acest lucru nu numai că economisește timp și efort, dar asigură și o coerență mai mare între diferitele platforme.

În concluzie, evoluția de la Objective-C și UIKit la Swift și SwiftUI reprezintă un pas important în dezvoltarea de aplicații iOS. Acesta reflectă angajamentul continuu al Apple de a îmbunătăți și simplifica procesul de dezvoltare a aplicațiilor, oferind în același timp dezvoltatorilor instrumentele de care au nevoie pentru a crea aplicații puternice, atractive și intuitive. Cu SwiftUI, viitorul dezvoltării de aplicații iOS pare a fi unul plin de posibilități și oportunități.

4.3 Combine

Combine este un framework foarte folosit în SwiftUI și foarte puternic, care aduce o interfață Swift declarativă pentru gestionarea evenimentelor care se petrec asincron. Acest framework introduce conceptul de Publishers și Subscribers. În Combine, Publisherii reprezintă o secvență de valori în timp. Ei emit evenimente, precum înnoiri ale datelor sau erori la Subscribers. Cei mai comuni Publisheri în networking includ *URLSessionDataTaskPublisher* - pentru a face cereri la rețea și *Just* pentru a emite o singură valoare. Publisherii conțin o multitudine de operatori care pot gestiona peste valorile obținute de la alți publisheri și le pot republica. Pe de altă parte, Subscribers sunt cei care primesc valorile și le gestionează pe măsură ce le primesc. Un lucru important este acela că Publishers pot doar să emită valori atunci când sunt cerute în mod explicit de Subscribers. Astfel, le dau controlul Subscriber-ului asupra ratei la care se primesc evenimente. În continuare, voi prezenta componentele principale din framework-ul Combine.

1. Publishers

În Combine, Publisherii reprezintă o secvență de valori în timp. Ei emit evenimente, precum înnoirea datelor sau a erorilor, către Subscribers. Doi dintre cei mai folosiți publisheri sunt *URLSessionDataTaskPublisher*, pentru a face cereri la rețea, și *Just*, pentru a emite o singură valoare.

2. Subscribers

Subscriberii gestionează valorile emise de publisheri. Aceștia pot primi date, pot trata erori și pot realiza, de asemenea, efecte secundare. Cele mai comune tipuri de Subscriberi sunt *sink*, *assign* și *onReceive*. Totodată, aceștia sunt și cei mai folosiți.

Subscriberul sink din Combine este folosit pentru a primi valori și pentru a completa evenimentele venite de la un Publisher. Este o metodă ușoară și la îndemână pentru a gestiona rezultatul unui publisher fără a fi necesară crearea unui subscriber personalizat.

Un exemplu de cum am utilizat eu acest subscriber în proiect:

```
func login() {
    userService.login(email: email, password: password)
        .receive(on: DispatchQueue.main)
        .sink { [weak self] completion in
            guard let self else { return }
            switch completion {
            case .failure(let error):
                self.loginCompletion.send(.failure(error))
            case .finished:
                break
            }
        } receiveValue: { [weak self] user in
            guard let self else { return }
            self.loginCompletion.send(.login)
        }
        .store(in: &bag)
}
```

Fig 4.3.1 sink

În figura de mai sus, am folosit subscriberul sink pentru a trata cele două cazuri posibile ale funcției de login: failure și completion. Dacă se găsea o eroare, în sink s-ar fi emis evenimentul de eroare. În caz contrar, s-ar fi luat valorile primite și s-ar fi trimis evenimentul de succes.

3. Operatori

Combine oferă un set bogat de operatori pentru a transforma și combina publisherii. Operatorii ne permit să modificăm datele, să filtrăm evenimentele, să gestionăm erorile sau să combinăm mai mulți publisheri într-un singur flux.

```
var subscriptions = Set<AnyCancellable>()
let publisher = [10, 20, 30].publisher
publisher
    .map({ $0 * 2 })
    .sink { print($0) }
    .store(in: &subscriptions)
//Output: 20, 40, 60
```

Fig 4.3.2 Folosirea operatorilor

În codul prezentat mai sus, ne cream un flux de date, pe care îl gestionăm în mai multe moduri. În primul rând, ne cream o mulțime (subscriptions) care va stoca obiectele de tip "AnyCancellable". Aceste obiecte sunt utilizate pentru a anula legăturile cu publisherii.

Apoi, ne cream un publisher, ce conține un vector de numere întregi. Asupra acestui publisher, aplicăm funcția `map({$0 * 2})`, ceea ce înseamnă că fiecare valoare emisă de publisher este înmulțită cu 2. Asupra acestei transformări, aplicăm funcția `sink { print($0) }`, ceea ce duce la crearea unui subscriber care va afișa fiecare valoare primită.

În final, asupra noului rezultat, aplicăm funcția `store(in: &subscriptions)`, care este echivalentă cu a spune că adăugăm obiectul la setul de subscripționari. Acest lucru este necesar pentru a menține legătura activă. Rezultatul final constă în emiterea de către publisher a unei serii de numere, fiecare dublată, iar rezultatele sunt afișate în consolă.

Multithreading

Multithreading este un proces de executare simultană a mai multor fire de execuție, unde execuția acestor fire poate fi de două tipuri diferite, cum ar fi Concurrent și Parallel. Putem să ne gândim la un fir de execuție ca la o listă de instrucțiuni, iar multithreading-ul ca la gestionarea mai multor liste de instrucțiuni în același timp.

Concurrency și Parallelism

- **Concurrency:** A rula un grup de sarcini în mod concomitent înseamnă că nu trebuie să așteptați ca o sarcină să se încheie înainte de a începe alta. Acestea pot fi executate simultan, pot avansa pe rând sau pot prezenta o combinație între cele două stări. Pentru utilizatorul final, acestea pot părea la fel ca și cum s-ar fi întâmplat în același timp.
- **Parallelism:** Un tip specific de concurență în care sarcinile sunt cu adevărat executate simultan.

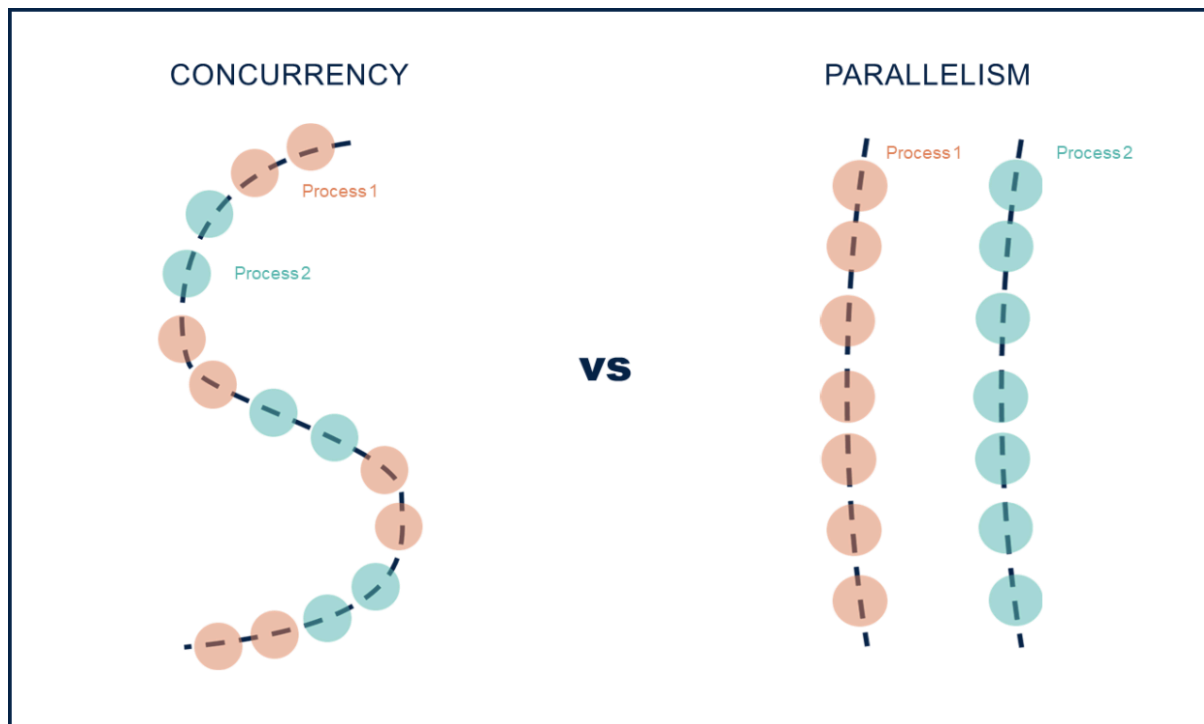


Figura 4.3.3 Concurrency vs Parallelism²

Înțelegerea Multithreading

Există doi termeni care trebuie înțeleși:

- Thread: unitatea independentă sau elementară a unui proces.
- Proces: programul care este în curs de executare; mai multe fire de execuție există într-un proces.

Exemplu din viața reală:

Să zicem că trebuie să realizăm 2 sarcini foarte importante într-o singură zi:

1. Să obținem un pașaport
2. Să realizăm o prezentare

Problema este că sarcina 1 ne obligă să mergem la un birou guvernamental extrem de birocratic care ne face să așteptăm 4 ore într-o coadă pentru a ne obține pașaportul. Între timp, sarcina 2 este solicitată de biroul nostru și este o sarcină critică. Ambele trebuie să fie finalizate într-o zi specifică.

Cazul 1: Execuție Secvențială

În mod obișnuit, vom conduce la biroul pașapoartelor timp de 2 ore, vom aștepta în coadă 4 ore, vom finaliza sarcina, vom conduce înapoi două ore, vom ajunge acasă, vom rămâne treaz încă 5 ore și vom realiza prezentarea.

² Imagine preluată din articolul "Concureny in Flutter", scris de DLT Labs, 8 septembrie 2021, URL: <https://medium.com/dlt-labs-publication/concurrency-in-flutter-dd0e14c4c930>

Cazul 2: Execuție Concomitentă

Dar suntem inteligenți. Ne pregătim în avans. Luam cu noi un laptop și, în timp ce așteptăm în coadă, începem să lucrăm la prezentarea cerută. Astfel, odată ce ajungem acasă, trebuie să lucrăm doar 1 oră în plus, în loc de 5.

În acest caz, ambele sarcini sunt realizate de noi, doar în bucăți. Am întrerupt sarcina pașaportului în timp ce așteptam în coadă și am lucrat la prezentare. Când a fost chemat numărul nostru, am întrerupt sarcina de prezentare și am trecut la sarcina pașaportului. Economia de timp a fost posibilă datorită posibilității de întrerupere a ambelor sarcini.

Cazul 3: Execuție Paralelă

Acum, deoarece suntem un oameni atât de isteți, evident că suntem la un nivel superior și avem un asistent. Deci, înainte de a pleca să începem sarcina pașaportului, îl sunăm și îi spunem să pregătească proiectul inițial al prezentării. Ne petrecem întreaga zi și terminăm sarcina pașaportului, ne întoarcem acasă, verificăm e-mailurile și găsim proiectul prezentării. A făcut o treabă destul de solidă și cu niște editări în încă 2 ore, o finalizăm.

Acum, deoarece asistentul nostru este la fel de isteț ca noi, a putut să lucreze independent la ea, fără a avea nevoie să ne întrebe constant pentru clarificări. Astfel, datorită abilității independente a sarcinilor, au fost realizate în același timp de doi executori diferiți.

Concluzia este următoarea:

- ★ Firele de execuție sunt independente și putem realiza mai multe operații în același timp.
- ★ Putem realiza multe operații în același timp, economisind timp.
- ★ Firele de execuție sunt independente, astfel încât nu afectează alte fire dacă apare o excepție într-un singur fir.
- ★ Indiferent dacă este vorba de concurență sau paralelism, scopul utilizării multitasking-ului este de a crește throughput-ul și performanța unei aplicații.

Arhitectura și Structura Aplicației

Atunci când lucrăm la aplicații iOS, este crucial să luăm în considerare arhitectura și structura aplicației. Acest lucru ajută la menținerea unui cod curat și scalabil. Un pattern arhitectural comun în Swift este Model-View-ViewModel (MVVM).

Putem structura aplicația astfel, de exemplu:

Networking

-> Services

--> UserService.swift etc.

-> API

--> UserAPI.swift etc.

Models

-> User.swift etc.

UI

-> ContentView.swift etc.

Resources

-> Assets, Colors, etc.

Networking

Networking este un aspect crucial în construirea aplicațiilor iOS care comunică cu servere sau API-uri de la distanță. Implică efectuarea de cereri HTTP, gestionarea răspunsurilor și procesarea datelor[8].

1. Introducere în HTTP

HTTP (Hypertext Transfer Protocol) este protocolul de bază folosit pentru comunicare pe web. Permite clienților (cum ar fi aplicațiile iOS) să trimită cereri la servere și să primească răspunsuri. Înțelegerea aspectelor de bază ale HTTP este esențială pentru lucrul cu cererile de rețea în aplicațiile iOS.

Metodele HTTP

HTTP suportă diverse metode pentru a realiza diferite acțiuni asupra resurselor. Metodele folosite în mod obișnuit includ:

- ➔ GET: Recuperează date de la un server. Este utilizat pentru preluarea resurselor.
- ➔ POST: Trimite date la un server pentru a crea o resursă nouă.
- ➔ PUT: Trimite date la un server pentru a actualiza o resursă existentă.
- ➔ DELETE: Trimite o cerere pentru a șterge o resursă de pe un server.

2. Serializare/Deserializare

Serializarea și deserializarea sunt procesele de conversie a datelor între diferite formate (cum ar fi JSON sau XML) și tipurile de date Swift (cum ar fi structurile sau clasele). Serializarea este folosită pentru a converti obiectele Swift într-un format potrivit pentru transmitere, în timp ce deserializarea convertește datele primite în obiecte Swift.

Exemplu de Serializare:

```
struct Person: Codable {  
    let name: String
```

```

let age: Int
}
let person = Person(name: "John", age: 30)
let encoder = JSONEncoder()
if let jsonData = try? encoder.encode(person),
    let jsonString = String(data: jsonData, encoding: .utf8) {
    print(jsonString)
}

```

Figura 4.3.5 Serializare

Exemplu de Deserializare:

```

let jsonString = """
{
  "name: "John",
  "age: 20
}
"""
let decoder = JSONDecoder()
if let jsonData = jsonString.data(using: .utf8),
    let person = try? decoder.decode(Person.self, from: jsonData) {
    print(person)
}

```

Fig 4.3.6 Deserializare

3. Apeluri API

Apelurile API (Interfețe de Programare a Aplicațiilor) implică realizarea de cereri către servere sau API-uri de la distanță pentru a prelua sau trimite date. În aplicațiile iOS, acest lucru se face în mod obișnuit folosind URLSession, un framework puternic furnizat de Apple.

```

struct API {
    static func fetchData() -> AnyPublisher<Data, Error> {
        let url = URL(string: "")!
        return URLSession.shared.dataTaskPublisher(for: url)
            .map(\.data)
            .eraseToAnyPublisher()
    }
}
let cancellable = API.fetchData()
    .sink { completion in
        // Gestionare completare, cum ar fi succes sau esec
    } receiveValue: { data in
        //Gestionare date primite
        print(data)
    }

```

Figura 4.3.7 Exemplu apel Api

4.4 Data persistence

Persistența datelor reprezintă un aspect crucial al construirii aplicațiilor iOS care necesită stocarea și recuperarea datelor între lansările aplicației. Ea implică salvarea și extragerea datelor către și de la diverse mecanisme de stocare disponibile în iOS.[9]

1. Keychain (Lanțul de chei)[10]

Keychain este un mecanism de stocare securizat în iOS care vă permite să stocați date sensibile precum parole, token-uri sau chei de criptare. Oferă o modalitate sigură și criptată de a stoca și recupera aceste date.

2. UserDefaults (Setările utilizatorului)[11]

UserDefaults este o modalitate simplă de a stoca și recupera setările utilizatorului și cantități mici de date. Furnizează o interfață ușoară și ușor de utilizat pentru stocarea persistentă. UserDefaults poate fi folosit pentru a stoca preferințele utilizatorului, setările și alte tipuri mici de date. Este adesea folosit pentru a salva setări precum tema aplicației, preferința de limbă sau preferințe specifice utilizatorului. Important: nu stocați date sensibile în UserDefaults.[13]

Exemplu:

```
let defaults = UserDefaults.standard
defaults.set(true, forKey: "darkModeEnables")
defaults.set("fish", forKey: "favoriteFood")
let darkModeEnables = defaults.bool(forKey: "darkModeEnabled")
let favoriteFood = defaults.string(forKey: "favoriteFood")
print("Dark mode enables: \(darkModeEnables)")
print("Favorite food: \(favoriteFood)")
```

Figura 4.4.1 Exemplu pentru UserDefaults

3. CoreData

CoreData este un framework puternic furnizat de Apple pentru gestionarea modelelor de date complexe și efectuarea persistenței datelor. Oferă o interfață orientată pe obiect și suportă relații, interogări și altele.

Cu CoreData, putem defini un model de date folosind entități, atribute și relații. Putem apoi efectua diverse operații precum crearea, preluarea, actualizarea și ștergerea datelor folosind contextul obiectului gestionat.

Concepte principale ale Core Data:

- ➔ Model de obiect gestionat: Modelul de obiect gestionat definește structura modelului dvs. de date folosind entități, atribute și relații. Reprezintă entitățile sub formă de clase sau structuri în codul dvs.

- Contextul obiectului gestionat: Contextul obiectului gestionat acționează ca o zonă temporară în memorie pentru a lucra cu obiectele gestionate. Reprezintă interfața între codul dvs. și depozitul persistent.
- Obiect gestionat: Obiectele gestionate reprezintă înregistrări individuale sau instanțe ale entităților din modelul nostru de date. Le putem privi ca pe obiecte care sunt persistate și recuperate din depozitul persistent.
- Entitate: O entitate este o descriere a unui tip specific de obiect sau tabel în modelul dvs. de date. Include attribute (proprietăți) și relații care definesc structura și comportamentul obiectelor.
- Atribut: Un atribut reprezintă o proprietate a unei entități și definește tipul de date al acelei proprietăți, cum ar fi șir, număr, dată sau boolean.
- Relație: O relație definește asociația dintre două entități. Poate fi o relație unu-la-unu, unu-la-mulți sau mulți-la-mulți, permițându-vă să stabiliți conexiuni între obiecte.
- Coordonatorul de depozit persistent: Coordonatorul de depozit persistent gestionează coordonarea dintre modelul de obiect gestionat și depozitul persistent. Se ocupă de persistența reală a datelor pe disc.

Operații principale cu Core Data:

- Creare: Putem crea obiecte gestionate noi prin inserarea lor în contextul obiectului gestionat. Specificați numele entității, atribuiți valori atributelor și stabiliți relații.
- Preluare: Putem prelua date din depozitul persistent pe baza unor criterii specifice folosind cereri de preluare. Cererile de preluare ne permit să recuperăm obiecte care se potrivesc cu filtrele specificate, să le sortăm și să limităm setul de rezultate.
- Actualizare: Pentru a actualiza un obiect gestionat, modificați attributele sau relațiile sale în cadrul contextului obiectului gestionat. După ce efectuăm modificările, salvăm contextul obiectului gestionat pentru a persista modificările în depozitul persistent.
- Ștergere: Pentru a șterge un obiect gestionat, îl eliminăm pur și simplu din contextul obiectului gestionat. Odată șters, obiectul va fi eliminat din depozitul persistent când se salvează modificările.

4. Permițiuni[12]

Permișiunile în iOS controlează accesul la diferite resurse și servicii, cum ar fi camera, microfonul, locația și altele. Înțelegerea modului de solicitare și gestionare a permișiunilor este esențială atunci când construiți aplicații care necesită acces la aceste resurse.

Exemplu:

```
func requestCameraPermission() {
    AVCaptureDevice.requestAccess(for: .video) { granted in
        if granted {
            print("Permisiunea pentru camera a fost acordata.")
        } else {
            print("Permisiunea pentru camera a fost refuzata.")
        }
    }
}
```

Figura 4.4.2 Permisuni in iOS

4.5 State management

Managementul stării este un aspect esențial al dezvoltării aplicațiilor iOS robuste. În SwiftUI, managementul stării joacă un rol crucial în gestionarea fluxului de date și în actualizarea interfeței utilizator în răspuns la schimbări. SwiftUI oferă diverse instrumente și tehnici pentru gestionarea stării, inclusiv modificatori de ciclu de viață, redesenare și framework-ul Combine.[15]

1. Modificatori de ciclu de viață (*onAppear* / *onReceive*)

În SwiftUI, modificatorii de ciclu de viață ne permit să executăm cod în puncte specifice pe durata ciclului de viață al unei vizualizări. Doi modificatori de ciclu de viață folosiți frecvent sunt *onAppear* și *onReceive*.

- ➔ Modificatorul *onAppear* = este folosit pentru a efectua acțiuni când o vizualizare apare pe ecran. Este util în special pentru inițializarea datelor, efectuarea cererilor API sau actualizarea stării când o vizualizare devine vizibilă.

```
struct ContentView: View {
    var body: some View {
        Text("Salutare")
        .onAppear {
            print("Vizualizarea a aparut!")
        }
    }
}
```

Fig 4.4.5.1 Modificatorul *onAppear*

- ➔ Modificatorul *onReceive* = ne permite să reacționăm la modificările într-un Publisher. Este folosit în mod obișnuit pentru a observa schimbările în proprietățile *@Published* din SwiftUI sau în publisher-ele Combine.

```
struct ContentView: View {
    @State var currentDate = Date()
```

```

let timer = Timer.publish(every: 1, on: .main, in: .common)
    .autoconnect()
var body: some View {
    Text("\(currentDate)")
        .onReceive(timer) { input in
            currentDate = input
        }
}
}

```

Figura 4.4.5.2 Modificatorul onReceive

2. Redesenare

În SwiftUI, framework-ul gestionează automat procesul de redesenare a vizualizărilor atunci când starea subiacentă se schimbă. Această abordare declarativă simplifică codul și asigură că interfața utilizator rămâne sincronizată cu datele.

În spatele scenei, SwiftUI se bazează pe un algoritm de diferențiere pentru a determina modificările dintre stările anterioare și curente. Apoi actualizează numai părțile afectate ale ierarhiei vizualizărilor, rezultând în redesenări eficiente.

3. Actualizarea stării

Actualizarea stării în SwiftUI se realizează în mod obișnuit folosind proprietăți împachetate precum `@State`, `@Binding`, `@ObservedObject` și `@EnvironmentObject`.

➔ `@State`: ne permite să declarăm o stare mutabilă în cadrul unei vizualizări. Când starea se schimbă, SwiftUI declanșează automat o redesenare a vizualizării afectate.

```

struct ButonRedare: View {
    @State private var seReda = false //Crearea starii

    var body: some View {
        Button(seReda ? "Pauza" : "Reda") { //Citirea starii
            seReda.toggle() //Scrierea starii
        }
    }
}

```

Figura 4.4.5.3 Actualizarea starii

➔ `@Binding`: stabilește o conexiune bidirecțională între o vizualizare părinte și una copil. Permite unei vizualizări copil să citească și să modifice o valoare deținută de vizualizarea părinte.

```

struct VizualizareParinte: View {

```

```

@State private var estePornit = false
var body: some View {
    VizualizareCopil(estePornit: $estePornit)
}
}
struct VizualizareCopil: View {
    @Binding var estePornit: bool

    var body: some View {
        Toggle("Intrerupator", isOn: $estePornit)
    }
}

```

Fig 4.4.5.4 Binding state

→ **@ObservedObject**: se folosește pentru a observa și a reacționa la schimbările într-un obiect extern care respectă protocolul **ObservableObject**.

```

class ModelTablouDeBord: ObservableObject {
    @Published var nume: String = "Nume initial"
}
struct VizualizareTablouDeBord: View {
    @ObservedObject var model = ModelTablouDeBord()

    var body: some View {
        VStack {
            Text(model.nume)

            Button {
                model.nume
            } label: {
                Text("Schimba numele")
            }
        }
    }
}

```

Fig 4.4.5.5 Observed state

→ **@EnvironmentObject**: ne permite să injectăm un obiect care respectă protocolul **ObservableObject** în ierarhia vizualizărilor, făcându-l accesibil vizualizărilor copil.

```

class DateUtilizator: ObservableObject {
    @Published var numeUtilizator= "Nume de Test"
}
struct VizualizareContinut: View {
    @EnvironmentObject var dateUtilizator: DateUtilizator

    var body: some View {
        Text("Bun venit, \(dateUtilizator.numeUtilizator)!")
    }
}

```



```

}
}

```

Fig 4.4.5.6 Environment Object state

4. Concepte de bază ale Combine

Combine este un framework puternic în Swift care oferă o abordare declarativă pentru gestionarea evenimentelor asincrone și pentru administrarea fluxurilor de date. SwiftUI se bazează pe Combine pentru programarea reactivă și gestionarea stării.

- ➔ **@Published**: este folosit pentru a marca proprietățile dintr-o subclasă ObservableObject ca fiind publicabile. Ea generează automat un publisher care emite o valoare ori de câte ori proprietatea marcată se schimbă.
- ➔ **@StateObject**: se utilizează pentru a crea un obiect observabil în cadrul unei vizualizări și pentru a asigura persistența acestuia pe durata actualizărilor vizuale. Este folosit în mod obișnuit pentru a crea și gestiona dependențe pentru o vizualizare specifică.

5. Sfaturi și trucuri de performanță[14]

- Utilizăm **@State** și **@Binding** pentru stări mici și locale: preferăm **@State** și **@Binding** pentru gestionarea stărilor mici și locale, deoarece oferă redesenări eficiente limitate la vizualizările afectate.
- Utilizăm **@ObservedObject** pentru stări mai mari și partajate: pentru stări mai mari sau partajate între mai multe vizualizări, utilizăm **@ObservedObject** pentru a observa modificările în mod eficient.
- Evităm redesenările inutile: trebuie să ne asigurăm că vizualizările noastre se redesenează doar atunci când este necesar. Utilizăm **Equatable** sau **Identifiable** pentru obiectele de stare pentru a ajuta SwiftUI să determine dacă o vizualizare necesită actualizare.
- Utilizăm cu atenție **@StateObject** și **@EnvironmentObject**: utilizăm **@StateObject** și **@EnvironmentObject** pentru a crea și gestiona dependențe, minimizând crearea inutilă a obiectelor.
- Combinăm operațiile asincrone cu Combine: valorificăm funcționalitățile puternice ale Combine pentru a gestiona operațiile asincrone, cum ar fi apelurile de rețea sau bazele de date, în mod eficient.
- Profil și optimizare performanță: utilizăm instrumentele de profilare ale Xcode pentru a identifica blocajele de performanță și pentru a optimiza aplicația SwiftUI în consecință.

4.6 Firebase

În general, aplicațiile de mobil nu își implementează un backend (spre deosebire de aplicațiile de web), așa că dezvoltatorii caută tot felul de alternative pentru un backend (sau o bază de date). Astfel, Google a inventat Firebase, un produs care îi ajută pe programatori să gestioneze și să dezvolte aplicațiile cu ușurință. Firebase este un tool foarte puternic, ce ține locul cu ușurință unui backend sau a unei baze de date. Această platformă oferă soluții pentru autentificare, pentru a stoca diferite tipuri de fișiere sau date și multe altele. De asemenea, nu este necesară nicio programare pentru Firebase. Este doar o bibliotecă pe care o importăm la începutul codului și apoi suntem liberi să ne folosim de tot ce ne oferă ea.

Firebase vine cu o multitudine de funcții și metode integrate, cum ar fi funcțiile de login, register, logout, delete account, etc. Toate aceste lucruri fac mult mai ușoară și eficientă folosirea Firebase-ului în loc să ne creăm propriul backend de la zero. O listă completă a serviciilor oferite de Firebase se poate vedea și în figura următoare:

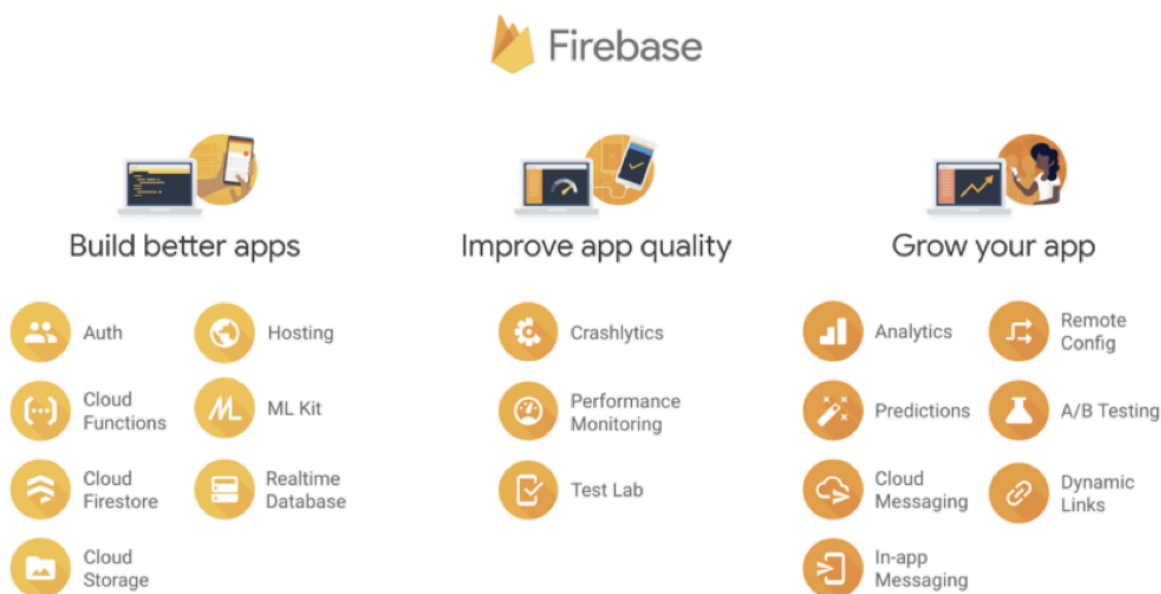


Figura 4.6.1 Lista serviciilor oferite de Firebase

Aplicatia TravelPal foloseste, din multitudinea de servicii oferite de Firebase, urmatoarele:

1. Autentificare
2. Baza de date Firestore

Autentificare

În prezent, majoritatea aplicațiilor, mai ales cele mobile, se construiesc în jurul unui utilizator. Deci, ele au nevoie de serviciul de autentificare, pentru a dispune datele din aplicație în funcție de preferințele utilizatorului, sau de diferite criterii, decide de dezvoltator.

Firebase Authentication este un serviciu complet de backend, ce conține biblioteci de UI ce sunt gata de utilizare. Este un serviciu complex, deoarece îi oferă dezvoltatorului multiple variante de autentificare: de la simpla autentificare cu email și parolă, la o autentificare bazată pe număr de telefon, pe "2 step authentication" și până la autentificare cu furnizori de identitate federați populari, precum Google, Facebook, etc.

Autentificare bazată pe email și parolă

Autentificarea bazată pe email și parolă a fost implementată și în proiectul TravelPal. Atunci când utilizatorul își face cont, el trebuie să își introducă numele, emailul, sexul și o parolă. În Firebase, utilizatorul primește un id unic. Atunci când acesta vrea să se conecteze la contul său, trebuia să își introducă doar emailul și parola, deoarece doar acestea două sunt necesare în Firebase pentru a-l identifica pe utilizator în baza de date.

Furnizorii de identitate federați

Firebase îi pune la dispoziție utilizatorului și opțiunea de a se conecta folosind un cont de socializare, precum Google, Facebook, Twitter, Apple, Yahoo, etc. Acest lucru face experiența utilizatorului mult mai plăcută.

Autentificarea prin numărul de telefon

Firebase permite autentificarea folosind numărul de telefon și, de asemenea, are serviciu de OTP integrat.

4.7 Figma

Figma este un tool destinat pentru a pune cap la cap design-ul unei aplicații (atât de web cât și de mobile), înainte ca dezvoltatorul să scrie cod. Figma poate fi folosit pentru multe alte tipuri de proiecte, însă, pentru aplicația TravelPal, am folosit Figma pentru a mă gândi cum mi-aș dori să arate aplicația mea, cum mi-aș dori să arate fiecare ecran (vor fi imagini cu rezultatul în capitolul care va urma).

5. Prezentarea aplicației TravelPal

5.1 Business logic

Aplicația TravelPal a fost creată cu scopul încurajării tuturor persoanelor să călătorească, indiferent dacă au partener de drum sau nu. Aplicația îi ajută pe cei care nu au partener de drum să-și găsească unul/mai mulți parteneri în cadrul ei. Ca și funcționalități, TravelPal are:

- Crearea unui cont
- Intrarea într-un cont deja existent
- Căutarea zborurilor în funcție de locul de plecare, de destinația dorită și de dată
- Selectarea unui zbor (și achitarea acestuia) sau selectarea itinerariului cu mașina
- Intrarea într-un chat asociat destinației unde și-a cumpărat zborul
- Trimiterea de mesaje în chaturile din care face parte
- Părăsirea unui chat din care face parte
- Salvarea unui bilet de avion în telefon sau trimiterea acestuia în altă aplicație

5.2 Structura aplicației TravelPal

Aplicația TravelPal folosește arhitectura MVVM (Model-View-ViewModel) și prezintă principiile de Clean Code. Toate fișierele aplicației se află într-un singur modul, cu același nume ca aplicația. Structurarea fișierelor intervine aici, deoarece, fiind un singur modul care conține toate fișierele aplicației, dacă acestea nu sunt sortate cumva, nu s-ar înțelege nimic din aceste fișiere.

Astfel, pe lângă fișierele care nu fac parte din niciun alt subfolder (SceneDelegate, AppDelegate, rootView, Info), în rest, am structurat fișierele după cum urmează:

- ➔ Fișierele care conțin call-urile la API-uri și deserializarea datelor, într-un folder dedicat numit API.
- ➔ Fișierele care conțin service-urile, cu call-urile către deserializările făcute la call-urile de API, într-un folder dedicat numit Services.
- ➔ Fișierele care conțin modelele de date/obiecte de care ne vom folosi atât la deserializare, cât și în ViewModele, într-un folder dedicat numit Model.
- ➔ Fișierele care conțin tot ce este legat de UI se află într-un folder cu același nume, iar ele sunt împărțite pe ecrane. De exemplu, pentru login avem un subfolder cu două fișiere, unul ce conține ViewModel-ul asociat loginului și un fișier ce conține View-ul asociat loginului.

→ Fișierele care conțin componente reutilizabile, imagini, culori, într-un folder dedicat numit Utils.

5.3 Funcționalități

Aplicația TravelPal se deschide cu un ecran de prezentare, numit "Splashscreen"(Figura 5.3.1). Acesta apare atunci când se deschide aplicația, pentru câteva secunde, în timp ce se încarcă scenele aplicației. În continuare, vom trata cazul în care utilizatorul este nou în aplicație.

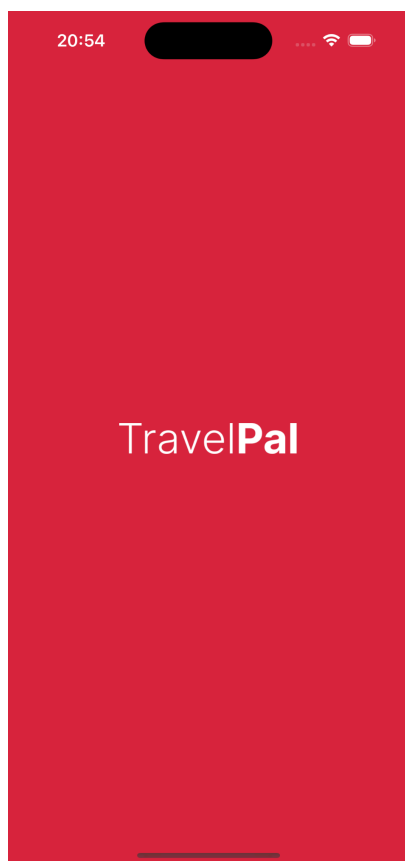


Fig. 5.3.1 Splashscreen

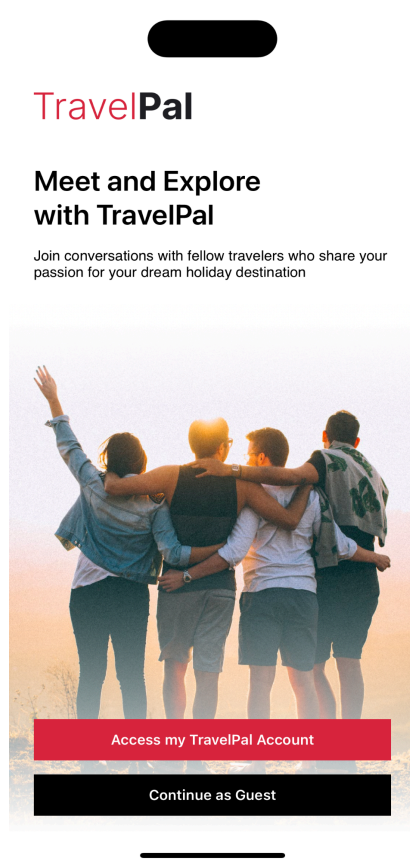


Fig. 5.3.2 Onboarding

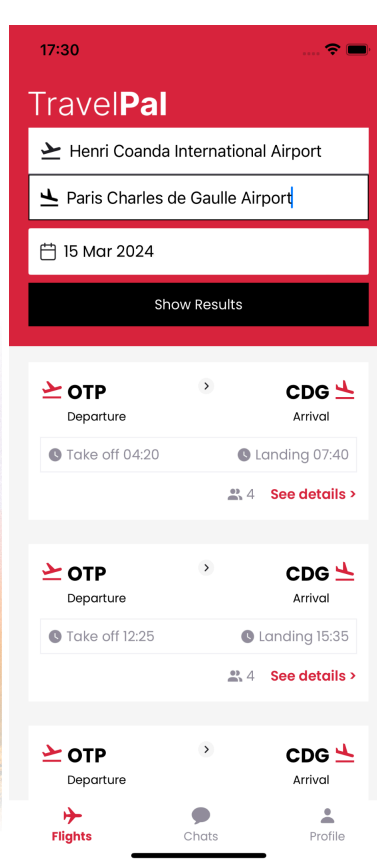


Fig. 5.3.3 Listare zboruri

Onboarding-ul este specific aplicațiilor mobile și poate avea unul sau mai multe ecrane, în care se prezintă, succint, prin text și imagini, scopul aplicației. Poate fi privit ca un scurt preview al scopului aplicației. Pentru aplicația TravelPal, am pregătit un Onboarding succint, dar reprezentativ (Figura 5.3.2). Acesta îl integrează pe utilizator către opțiunile aplicației: poate intra în contul său (în caz că nu are, există opțiunea să și-l creeze) sau poate continua ca "Guest", însă va putea accesa un număr redus de funcționalități ale aplicației.

De aici, se formează două cazuri: utilizatorul se autentifică/își creează un cont și astfel are acces la toate funcționalitățile aplicației, având o experiență completă în aplicație. Cel de-al doilea caz se referă la faptul că utilizatorul să nu intre în cont, dar să acceseze aplicația. Astfel, acesta va putea accesa doar pagina de căutare a zborurilor și listarea acestora. Dacă va dori să acceseze

detalii despre un zbor, pagina contului său sau pagina de conversații, va apărea un mesaj în care i se cere intrarea în cont pentru a continua.(Fig. 5.3.3)

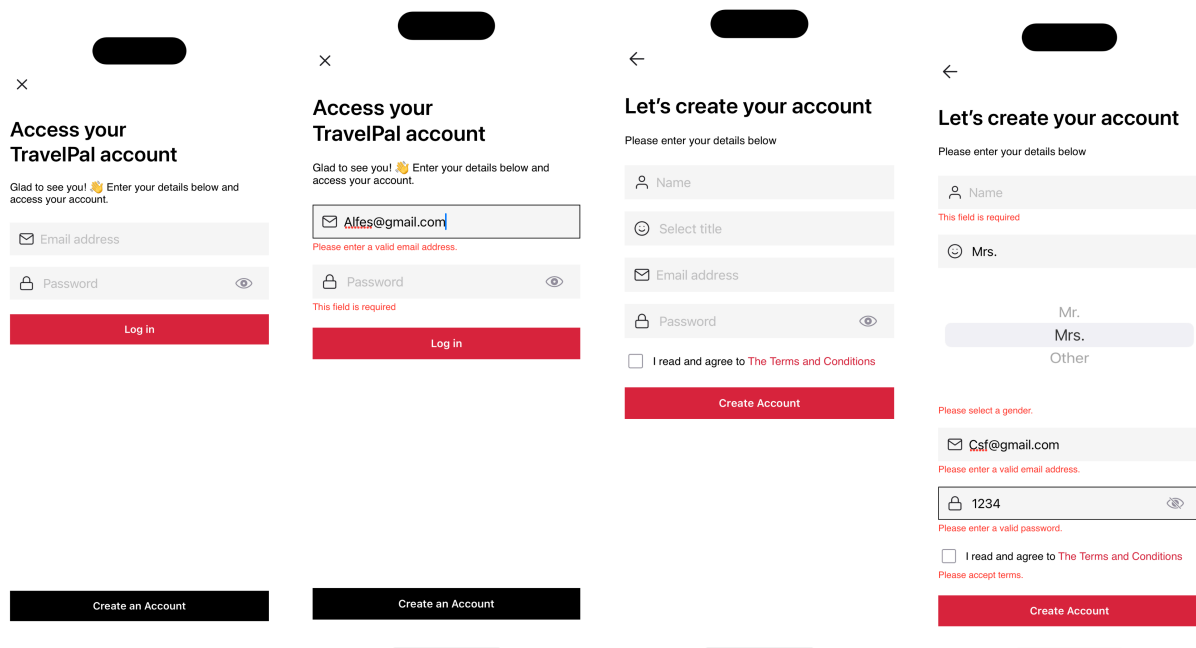


Fig. 5.3.4 Ecran de login Fig. 5.3.5 Erori Fig. 5.3.6 Ecran de înregistrare Fig 5.3.7 Erori

Pe de altă parte, dacă utilizatorul alege să intre în cont, acesta va fi trimis, inițial, pe pagina de conectare la un cont deja existent (Fig. 5.3.4). Aici, utilizatorul este întâmpinat cu un mesaj de bun venit și este rugat să-și introducă emailul și parola. Există posibilitatea ca utilizatorul să-și ascundă sau să nu ascundă parola. De asemenea, dacă acesta introduce o adresă de email sau o parolă care nu apar în baza de date de la Firebase Authentication, vor apărea mesaje de eroare conform cu problema. Este de menționat și faptul că mesaje de eroare apar și dacă acesta încearcă să apese pe butonul de "Log in" dacă nu a completat ambele câmpuri (Fig. 5.3.5).

Tot în acest ecran, există un buton care îi permite utilizatorului să navigheze către pagina de creare cont (Fig. 5.3.6). Această funcționalitate se adresează utilizatorilor noi sau celor care vor să-și schimbe contul. În această nouă pagină, utilizatorului îi sunt cerute mai multe date personale și, de asemenea, îi este cerut să aprobe termenii și condițiile aplicației (termeni și condiții care au fost generate pe internet). Utilizatorul poate vedea termenii și condițiile, cu o singură apăsare a textului. Ca date personale, TravelPal nu necesită multe informații despre utilizatori, ci doar numele (care poate fi și porecla din aplicație), adresa de email, parola și titlul utilizatorului (bărbat, femeie, etc). Numele și formula de adresare sunt cerute doar pentru a dispune datele mai frumos în aplicație, în funcție de ce a scris utilizatorul în aceste două câmpuri. De exemplu, pentru nume va fi afișată o poză pe pagina contului cu o față, dacă

utilizatorul a ales că este fată. Voi prezenta mai multe utilizări ale acestor date pe măsură ce prezint aplicația.

Revenind la pagina de creare a unui cont, sunt aceleași validări ca la pagina de conectare (Fig. 5.3.7). De asemenea, un lucru care trebuie menționat este acela că, odată ce utilizatorul a trecut pentru prima dată prin Onboarding, următoarea dată când deschide aplicația nu îi va mai apărea acesta. Onboarding-ul apare doar la instalare și la prima deschidere a aplicației. În rest, se va deschide ori în pagina de listare a zborurilor, unde pot să fie căutate rute, ori în pagina de conectare.

După realizarea autentificării, utilizatorul este întâmpinat de aceeași pagină de listare a zborurilor (Fig. 5.3.3). Acesta are opțiunea de a căuta aeroportul de plecare, cât și cel de sosire și, de asemenea, poate selecta și o dată. Dacă există zboruri pentru criteriile de selecție, se vor afișa, ca în figura prezentată mai devreme. În caz contrar, se va afișa un mesaj corespunzător pe ecran (Fig. 5.3.8). Mai departe, utilizatorul se poate uita în lista tuturor rutelor găsite, astfel încât să găsească zborul cel mai bun pentru el. Odată cu apăsarea pe butonul “See Details”/“Vezi detalii”, utilizatorul intră într-un nou ecran, unde poate vedea mai multe date despre zbor și, de asemenea, tot aici va fi întâmpinat de un modal explicativ pentru ceea ce se va prezenta în următoarea secțiune. (Fig. 5.3.9) Aici, utilizatorul poate alege să călătorească cu avionul către destinația selectată, sau poate selecta să meargă cu mașina. Am inclus și opțiunea de a merge cu mașina, tocmai pentru a-i încuraja și pe pasionații de condus să folosească aplicația "TravelPal" și să se conecteze cu alți parteneri de călătorie.

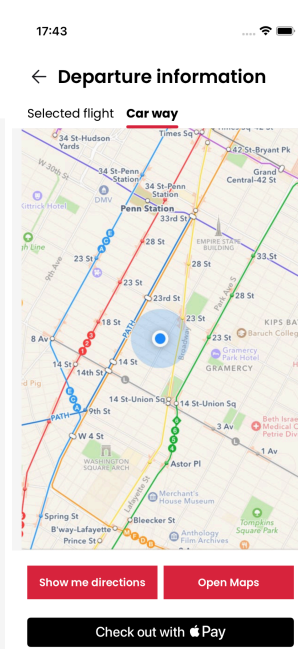
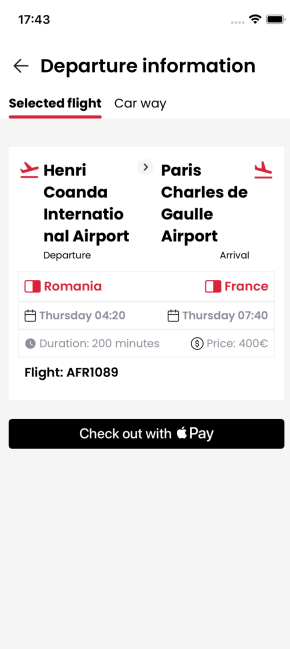
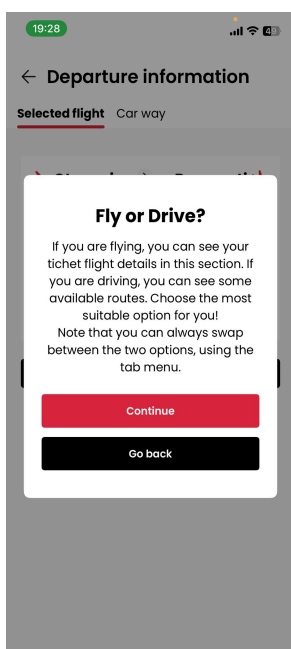
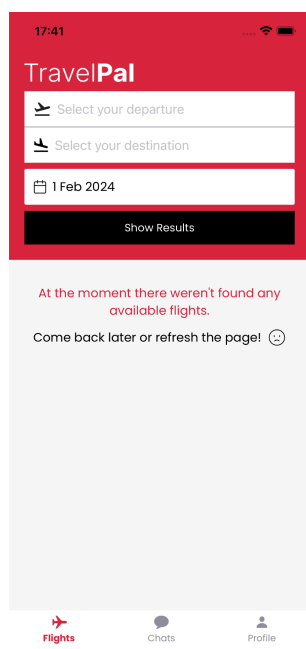


Fig.5.3.8 Nu s-au găsit zboruri; Fig.5.3.9 Modal Fig.5.3.10 Bilet avion Fig.5.3.11 Cu mașina

Să ne concentrăm puțin atenția asupra acestei secțiuni. După dispariția modalului informativ, suntem întâmpinați de un meniu cu 2 tab-uri de selecție: unul pentru biletul de avion, iar celălalt pentru modalități de a merge cu mașina către destinația selectată.

Dacă selectăm tab-ul ce conține biletul de avion (la încărcarea ecranului, el vine preselectat), vom vedea detalii despre zborul pe care l-am ales la pasul anterior (Fig. 5.3.10) și anume: locul de plecare, destinația, data, ora, durata, prețul și numărul zborului. Dacă utilizatorul dorește să aleagă această variantă de a călători, va continua prin apăsarea butonului de "Check out with Pay". Ce presupune acest buton? Simularea plății folosind Apple Pay (plată de sistem oferită de Apple pentru utilizatorii de iPhone).

Pe de altă parte, dacă selectăm tab-ul de mers cu mașina, vom fi întâmpinați cu cele 3 butoane din partea de jos a ecranului (Fig. 5.3.11). Să le luăm pe rând:

- Butonul cu textul “Show me directions” va deschide un modal de sistem (Fig. 5.3.12) pentru a cere permisiunea utilizatorului ca aplicația TravelPal să îi acceseze locația. Dacă utilizatorul nu acceptă, vor fi preluate coordonatele orașului București. Dacă acesta permite folosirea locației, se va vedea harta lumii, însoțită de un posibil traseu de urmat cu mașina către destinație (Fig. 5.3.13).
 - La deschiderea acestei hărți, există și un buton cu textul “Show me directions”, care, la apăsarea lui, va deschide un modal care va prezenta toate direcțiile de orientare, începând de la locul de plecare și până la destinație (Fig. 5.3.14).
- Butonul cu textul “Open Maps” va deschide aplicația dată de Apple pe iPhone, numită chiar Maps, cu destinația selectată de utilizatorul în aplicația TravelPal, preselectată (Fig. 5.3.15).
- Butonul cu textul “Check out with Pay”, care are același rol ca și cel aflat în tab-ul cu biletul de avion.

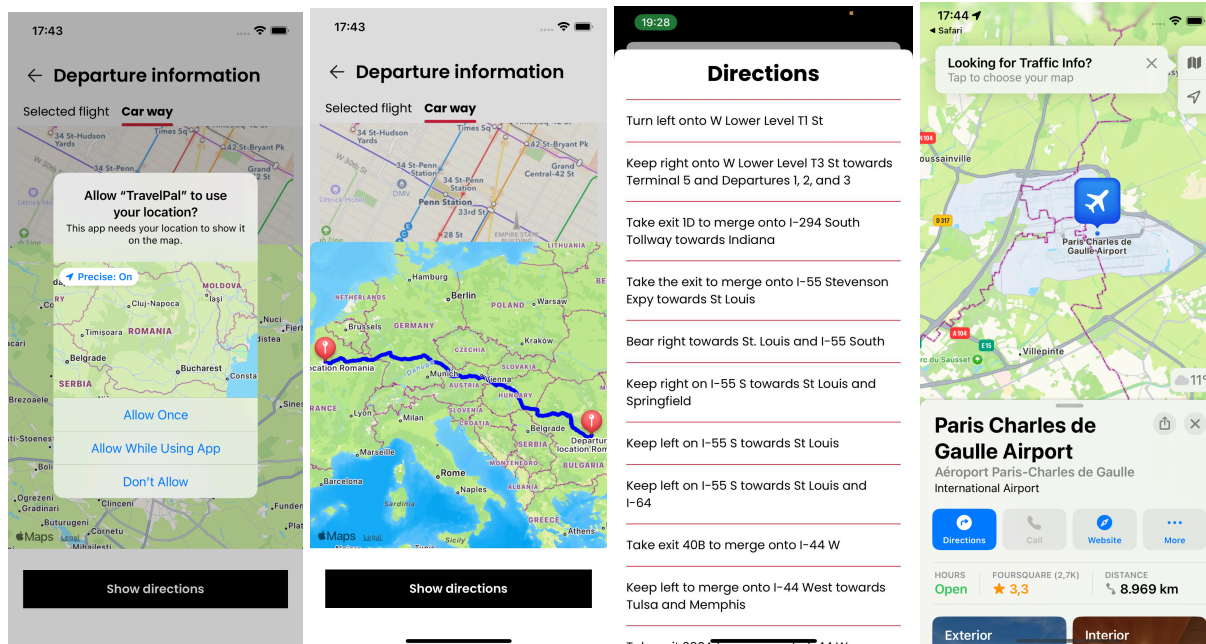


Fig.5.3.12 Permisuni 5.3.13 Hartă 5.3.14 Direcții de deplasare Fig.5.3.15 Apple Maps

În continuare, atunci când utilizatorul alege cum va călători (cu avionul sau cu mașina) și va apăsa pe butonul de Check out, se va deschide un bottom sheet, care are rolul de a simula plata prin Apple Pay (Fig. 5.3.16). La acest pas, vor fi prezentate câteva date ne-personale, tocmai pentru a simula o plată reală. Dacă aș fi vrut să realizez o plată reală, erau necesare mult mai multe documente și certificări de la Apple, lucruri care se obțin atunci când vrei să lansezi, real, o aplicație pe App Store.

Mai departe, când utilizatorul apasă pe butonul de “Pay”, aplicația va intra într-o stare de “Loading”, de încărcare, apoi va prezenta ecranul cu opțiuni de cazare. În primul rând, la prima încărcare a ecranului, se va prezenta o notificare (un “toast”) care are scopul de a-l informa pe utilizator că plata s-a efectuat cu succes. Tot în acest ecran, dacă biletul de avion a putut fi pozat de către aplicație, va exista un buton de share, cu opțiunea ca utilizatorul să-și salveze biletul de avion în telefon. De asemenea, există și un text informativ despre ce urmează în aplicație și ce trebuie/ poate să mai facă utilizatorul pentru a ajunge în mult-doritul chat.

Am încărcat câteva poze care reprezintă câteva opțiuni de cazare, care pot fi regăsite, de asemenea, și pe site-ul destinat rezervării de cazare, numit Airbnb. Pentru a accesa și mai multe cazări, textul subliniat “here” (Fig. 5.3.17) din acest ecran îl va redirecționa pe utilizator către site-ul de la Airbnb, cu destinația și data preselectate (așa cum le selectase acesta în aplicația TravelPal) (Fig. 5.3.18).

Butonul cu textul “Join chat” va încărca prezentarea modalului de confirmare a călătoriei (Fig. 5.3.19), care are un buton ce-l va adăuga pe utilizator în chat, alături de restul utilizatorilor care au ales să călătorească în aceeași destinație, prin intermediul aplicației TravelPal.

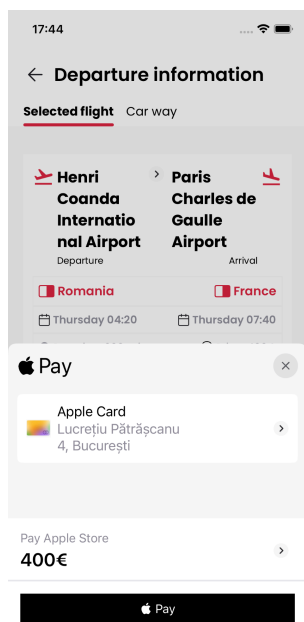


Fig.5.3.16 Plata

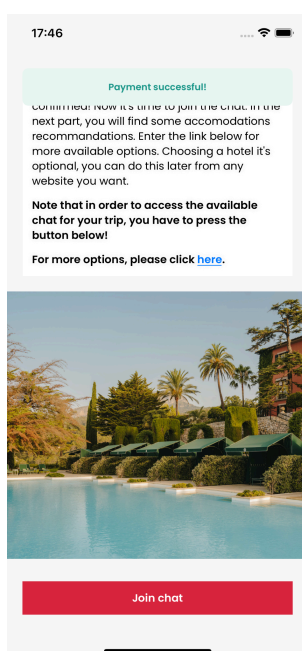


Fig.5.3.17 Confirmarea plății

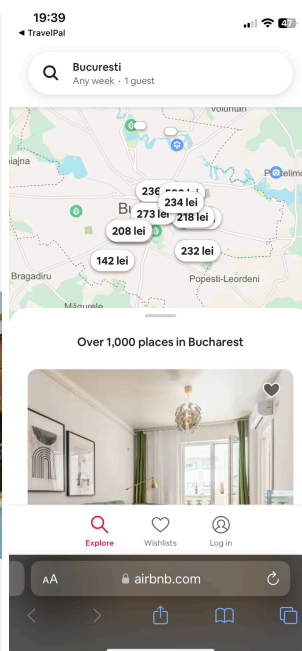


Fig.5.3.18 Cazări

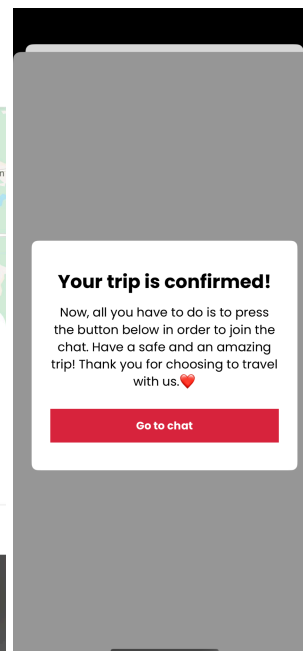


Fig.5.3.19 Redirecționare

În continuare, la apăsarea butonului de “Go to chat”, utilizatorul este redirecționat către chat-ul cu destinația pentru care și-a cumpărat un bilet de avion / a confirmat cu mașina. În prima fază, când utilizatorul abia intră în aplicație, după crearea contului, acesta nu face parte din niciun chat. Astfel, în tab-ul destinat chat-urilor, se va afișa un mesaj corespunzător și informativ, din care utilizatorul va ști că după achitarea unei călătorii, acolo îi vor apărea toate chat-urile din care va face parte (Fig. 5.3.20).

Pe de altă parte, după achiziționarea uneia sau mai multor călătorii, pe tab-ul de Chats vor fi afișate toate conversațiile din care acesta face parte și, de asemenea, se va regăsi, pentru fiecare conversație, numărul de persoane participante la acea conversație care, de fapt, înseamnă chiar numărul de persoane care merg, sunt sau au fost în acea destinație (Fig. 5.3.21).

La apăsarea pe o conversație, aceasta se va deschide și vor fi afișate toate mesajele care sunt, în timp real, sau au fost trimise în acea conversație și, totodată, posibilitatea ca și utilizatorul să trimită unul sau mai multe mesaje (Fig. 5.3.22). De asemenea, se poate observa în partea dreapta-sus a ecranului, un buton al cărui rol este de a-i oferi utilizatorului posibilitatea să

părăsească acea conversație. Dacă acesta alege să o părăsească, nu o va mai putea accesa până când nu își cumpără un nou bilet către aceeași destinație (Fig. 5.3.23).

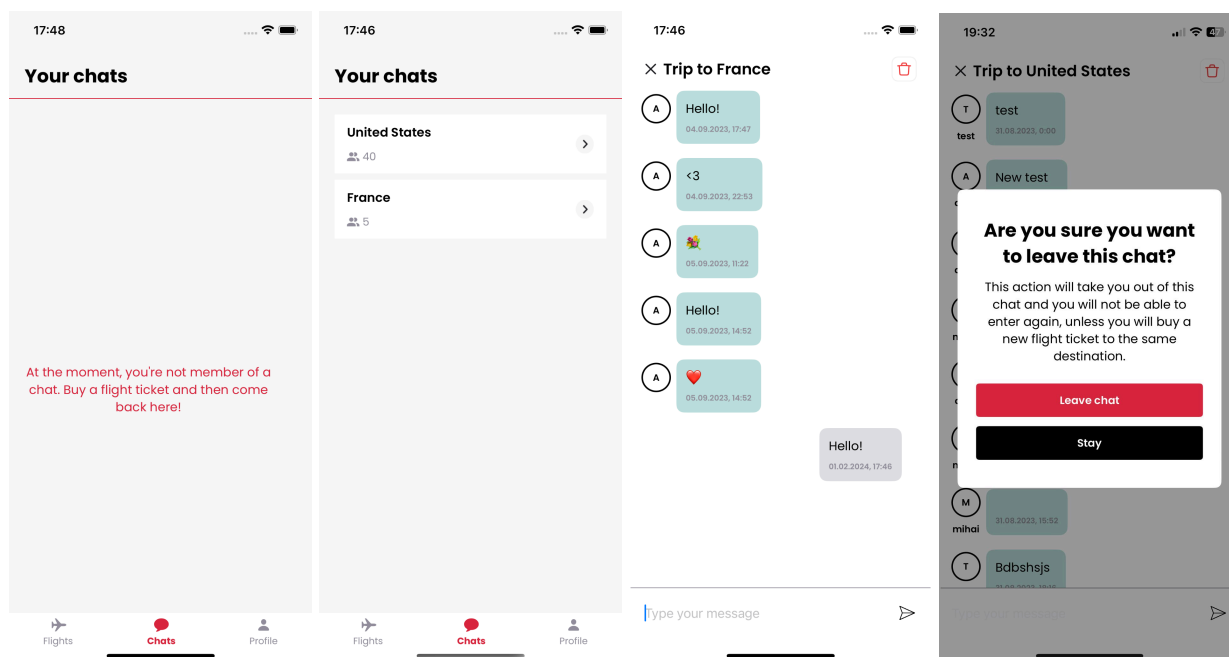


Fig.5.3.20 Fără chat Fig.5.3.21 Cu chat Fig.5.3.22 Conversație Fig.5.3.23 Părăsirea chat-ului

Ultima parte a aplicației TravelPal este alcătuită din pagina de profil, unde sunt dispuse câteva interacțiuni ale utilizatorului cu aplicația. În primul rând, este afișată inițiala din numele pe care utilizatorul l-a introdus la crearea contului, este afișat un mesaj de primire și, în continuare, sunt mai multe butoane, fiecare oferindu-i utilizatorului câte o nouă experiență (Fig. 5.3.24).

- Butonul de “Termeni și condiții” îi va prezenta utilizatorului, exact ca pe pagina de creare cont, Termenii și Condițiile aplicației (termeni care au fost generați online) (Fig. 5.3.25).
- Butonul de “Contactează-ne” va deschide aplicația “Mail” oferită de Apple, iar la destinatar va fi adresa mea de mail pre-completată, deoarece orice plângere despre aplicația TravelPal va ajunge la mine (Fig. 5.3.26).
- Butonul de “Schimbă limba” va deschide setările telefonului și îi va permite utilizatorului să aleagă o limbă dintre cele două disponibile în aplicație, română și engleză. Apoi, după selectarea unei limbi, toate textele din aplicație vor fi conforme cu limba selectată (Fig. 5.3.27).

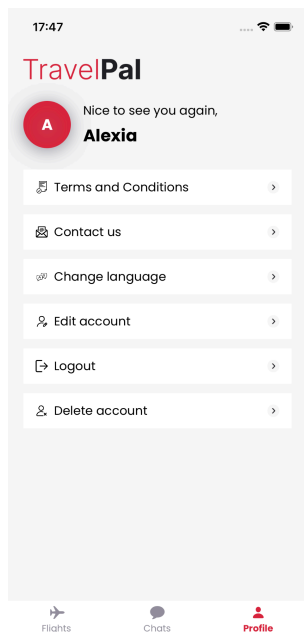


Fig.5.3.24 Profil

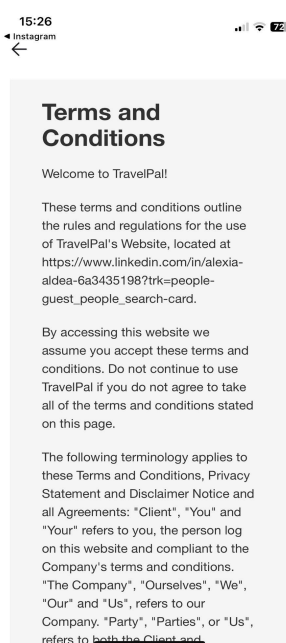


Fig.5.3.25 Termeni, condiții

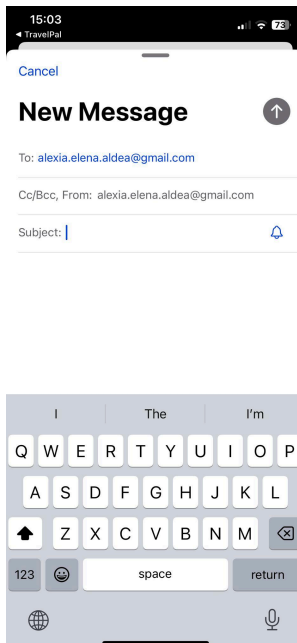


Fig.5.3.26 Contact

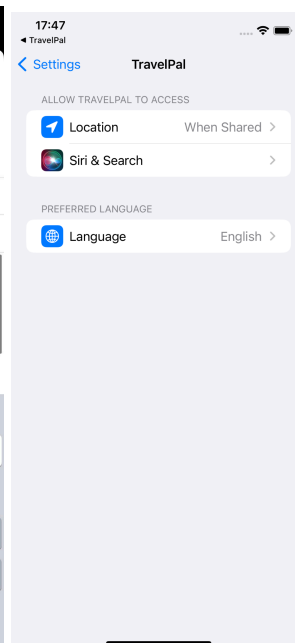


Fig.5.3.27 Limbă

- Butonul de “Editează cont” îi va permite utilizatorului să-și editeze numele și/sau genul, informații care au fost introduse la crearea contului (Fig. 5.3.28). În partea de jos a ecranului, dacă acesta nu a realizat nicio schimbare asupra datelor sale, butonul este dezactivat. Însă, dacă utilizatorul a modificat ceva, butonul devine activ, iar la apăsarea lui, datele utilizatorului vor fi schimbate.
- Butonul de “LogOut” va prezenta un modal în care utilizatorul va fi întrebat dacă este sigur că vrea să se delogheze (Fig. 5.3.29). Dacă utilizatorul este sigur și alege să se delogheze, aplicația se va întoarce la starea inițială, aceea de utilizator nelogat, în care multe informații (chat, pagina de profil, detalii zbor etc.) nu vor fi afișate, ci doar opțiunile de a intra în cont sau de a-și crea un cont și, de asemenea, se va afișa listarea tuturor zborurilor, cu opțiunea căutării unor zboruri după aceleași filtre (plecare, destinație, dată).
- Butonul de “Ștergere cont” va afișa și el, la rândul său, un modal informativ (Fig. 5.3.30), doar că, de această dată, dacă utilizatorul este sigur și alege să-și șteargă contul, toate informațiile despre acest cont vor fi pierdute, inclusiv chaturile din care face parte utilizatorul.

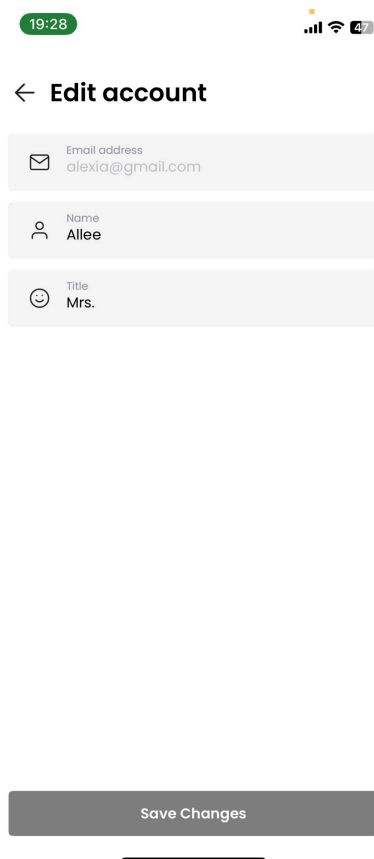


Fig 5.3.28 Editare cont

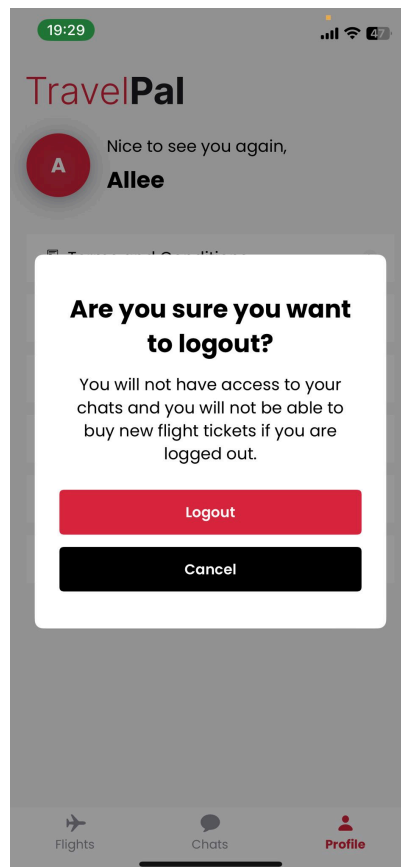


Fig. 5.3.29 Delogare

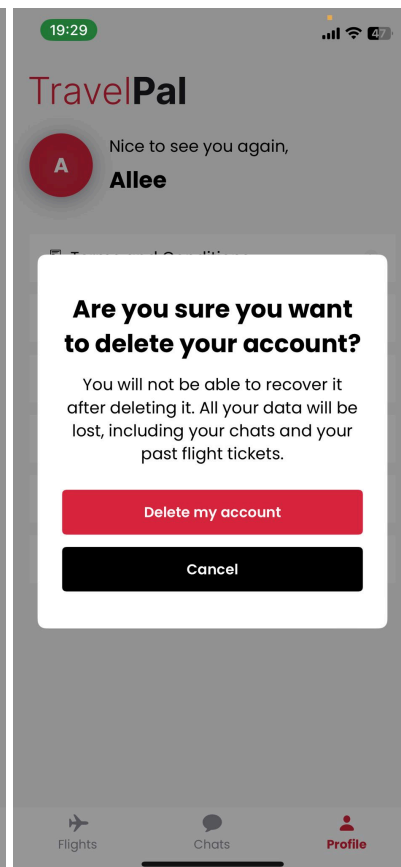


Fig. 5.3.30 Ștergerea contului

5.4 Posibile direcții de dezvoltare

Dacă vorbim despre posibile direcții de dezvoltare ale aplicației TravelPal, trebuie menționat faptul că aplicația are un mare potențial de a fi dezvoltată tot mai mult. Pornind de la design și până la funcționalități noi adăugate, direcțiile de dezvoltare ale aplicației sunt numeroase. În continuare, voi prezenta cele mai importante direcții de dezvoltare.

Pagina de “My account”

Această pagină poate fi mult dezvoltată în sensul că se pot adăuga mai multe funcționalități care să îi permită utilizatorului să-și personalizeze cât mai mult experiența în aplicația TravelPal. De exemplu, funcționalități precum "Resetează parola", adăugarea unei poze de profil, editarea numelui/ a contului sunt câteva exemple care pot fi adăugate în viitor, în aplicație.

Pagina de “Flights list”

Aici, se pot adăuga mai multe filtre de căutare, cum ar fi aspecte generale despre o destinație (munte, mare etc). De asemenea, pot fi afișate mai multe detalii despre fiecare zbor și fiecare destinație.

Pagina de “Chats”

Această funcționalitate poate fi cel mai mult exploatată. De exemplu, ar putea fi îmbunătățită prin a adăuga posibilitatea de a trimite poze în chat, de a reacționa la mesaje, de a răspunde la un anumit mesaj (reply) și multe altele. Tot aici, ar putea fi introdusă funcționalitatea de a adăuga prieteni, de a vorbi cu prietenii pe conversații private.

Alte pagini

O altă direcție de dezvoltare a aplicației TravelPal se conturează prin adăugarea unei pagini cu o listă de cazări, în funcție de destinația aleasă. În prezent, în aplicație există doar un buton care îl duce pe utilizator către site-ul “airbnb”, cu data și destinația preselectate în funcție de ce a selectat utilizatorul atunci când și-a cumpărat biletul de zbor. Această funcționalitate ar putea fi îmbunătățită prin aducerea acelor cazări în aplicația TravelPal, astfel încât utilizatorul să își poată achiziționa în cadrul acestei aplicații atât bilet de zbor, cât și o cazare.

6. Concluzii

Aplicația TravelPal a pornit de la problema pe care am întâlnit-o tot mai des în societate, aceea a oamenilor singuri, care nu au un partener constant de drum, de călătorit. Astfel, oamenii care nu sunt într-o relație nu pot călători când își doresc, dacă nu au cu cine să meargă. Am venit cu aplicația TravelPal tocmai pentru a combate acest lucru și pentru a le oferi și lor, în mod egal, șansa de a-și găsi parteneri de călătorit, chiar după ce își cumpără bilet de zbor. Astfel, sunt încurajate două lucruri: în primul rând călătoritul, iar în al doilea rând socializarea cu oameni noi. Având în vedere faptul că utilizatorul este adăugat automat într-o conversație cu alți utilizatori (cel mai probabil necunoscuți), el este nevoit să socializeze cu ei, pentru a se întâlni la destinație și pentru a se plimba împreună. Astfel, își îmbunătățește și abilitatea de a comunica cu oamenii necunoscuți și de a-și face prieteni noi.

Am ales să implementez această aplicație ca fiind una mobilă, deoarece consider că este cel mai ușor pentru utilizatori să utilizeze o astfel de aplicație pe telefon, de oriunde s-ar afla. În acest mod, utilizatorul deschide cu ușurință aplicația oricând, din orice loc și poate verifica ce rute există, când și poate verifica mesajele noi din conversațiile din care face parte (și poate da și el, la rândul lui, mesaje).

În concluzie, aplicația TravelPal poate ajuta o categorie de oameni care devine tot mai numeroasă, cei singuri, fără partener de călătorit. TravelPal poate fi destinată oricărui tip de utilizator major, indiferent de locul de muncă sau interese, deoarece scopul ei este să unească mai mulți oameni cu aceeași dorință, să creeze noi legături frumoase și să dea șansa oricui să călătorească, fără să depindă de cineva.

Bibliografie

- [1] Becky Pokora, “Travel Trends 2023: 49% Of Consumers Plan To Travel More In 2023”, în *Forbes Media LLC* (Aug. 2023), URL:
<https://www.forbes.com/advisor/credit-cards/inflation-has-not-deflated-americas-travel-plans-2023/>
- [2] Angela, “14 BENEFITS OF TRAVELLING (& Easy Ways To Enhance Your Travel Experience)”, în *Global Gadding* (Iul. 2023), URL:
<https://www.globalgadding.com/benefits-of-travelling/>
- [3] Maciej Markiewicz, “Mobile Application Architecture – Why Is It So Important?”, în *Netguru* (Oct. 2023), URL:
<https://www.netguru.com/blog/mobile-application-architecture-why-is-it-so-important>
- [4] Ben Myers, “Create a Scalable SwiftUI MVVM Project”, în *Better Programming* (Sep. 2022), URL:
<https://betterprogramming.pub/create-a-scalable-swiftui-mvvm-project-f9afe3c2fcbf>
- [5] Autor necunoscut, “Principiile SOLID, toate cele 5 principii solid RO”, în studocu, URL:
<https://www.studocu.com/row/document/universitatea-tehnica-a-moldovei/tehnici-si-mecanize-d-e-proiectare-a-sabloanelor/principiile-solid-toate-cele-5-principii-solid-ro/29033365>
- [6] Apple Company, “Xcode”, în documentația oficială de la Apple, URL:
<https://developer.apple.com/documentation/xcode>
- [7] Apple Company, “SwiftUI”, în documentația oficială de la Apple, URL:
<https://developer.apple.com/documentation/swiftui/>
- [8] Autor necunoscut, “Networking with Combine and SwiftUI”, în Peter Friese (Ian. 2022), URL: <https://peterfriese.dev/posts/swiftui-combine-networking-gettingstarted/>
- [9] Apple Company, “SwiftUI”, în documentația oficială de la Apple, URL:
<https://developer.apple.com/documentation/coredata>
- [10] Apple Company, “SwiftUI”, în documentația oficială de la Apple, URL:
https://developer.apple.com/documentation/security/certificate_key_and_trust_services/keys/storing_keys_in_the_keychain
- [11] Apple Company, “SwiftUI”, în documentația oficială de la Apple, URL:

<https://developer.apple.com/documentation/foundation/userdefaults>

[12] Apple Company, “SwiftUI”, în documentația oficială de la Apple, URL: https://developer.apple.com/documentation/corelocation/requesting_authorization_to_use_location_services

[13] Autor necunoscut, “Reading and writing basics: UserDefaults”, în Hacking with Swift, URL: <https://www.hackingwithswift.com/read/12/2/reading-and-writing-basics-userdefaults>

[14] Apple Company, “Model data”, în documentația oficială de la Apple, URL: <https://developer.apple.com/documentation/swiftui/model-data>

[15] Autor necunoscut, “A guide to SwiftUI’s state management system”, în Swift with Sundell (Iul. 2020), URL: <https://www.swiftbysundell.com/articles/swiftui-state-management-guide/>